Applied Robotics (FRTN85)

LECTURE 2

BJÖRN OLOFSSON, DEPT. AUTOMATIC CONTROL, LTH

Today's Lecture

- Course administration
- Translations and rotations of rigid bodies
- Robot kinematics
  - Forward kinematics
  - Denavit-Hartenberg (DH) convention

LUND UNIVERSITY

Course Administration

Course Administration

- Mandatory hands-on exercise in RobotLab and work on first RobotStudio exercise last week
  - If you have not completed the hands-on exercise in the lab, ***please contact me as soon as possible***
- ***Sign-up for computer exercises***
  - Continue with RobotStudio exercises and demonstrate them in the RobotLab after approval from TA
  - Get started with Peter Corke's robotics toolbox in Matlab / Python (later part of the week)
  - Can sign-up for one session per week in Canvas, but most welcome to attend additional sessions

LUND UNIVERSITY

Overview of Assignments in the Course

Course Week	RobotStudio Exercises	Kinematics and Dynamics Exercises (FRTN85 only)	Project	Hand-in Exercise
1	x			
2	x	x		
3	x	x	Introduction	
4	x	x	x	x
5	x	x	x	x
6		x	x	x
7			x	x
8			Presentation	Deadline

Preparation for Hand-in Exercise

LUND UNIVERSITY

Robotics in this Course

Recap from Previous Lecture

- Introduction to robotics and application areas – many examples of both robot designs and robot tasks
- Robotics is a multidisciplinary topic
- System aspects important for solving robotic tasks

LUND UNIVERSITY

Industrial Robot

Industrial robot as defined by ISO 8373:

An automatically controlled, reprogrammable, multipurpose manipulator programmable in three or more axes, which may be *either fixed in place or mobile* for use in industrial automation applications.

LUND UNIVERSITY

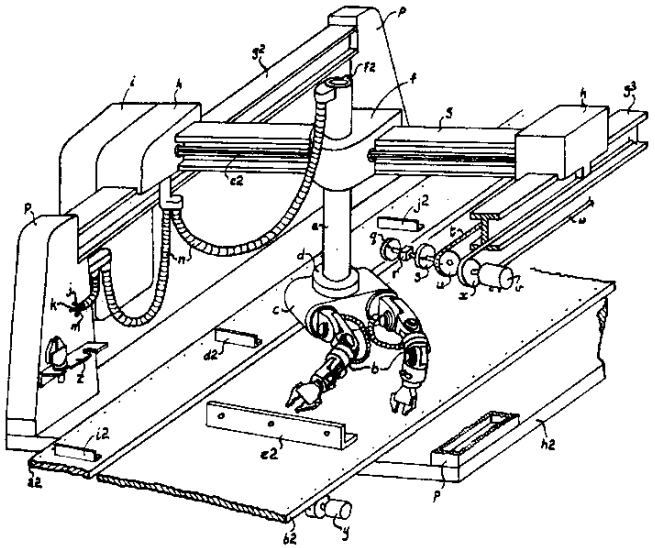
What is a Robot?

A robot is a **reprogrammable**, **multifunctional** manipulator designed to move material, parts, tools, or specialized devices through *variable programmed motions* for the performance of a variety of tasks.

The word “robot” is derived from the Czech word “robota“ meaning heavy, monotonous or forced labour work. It was introduced in the play R.U.R. (Rossums Universal Robots) written by Karel Capek around 1920 with first performance in Prague 1921.

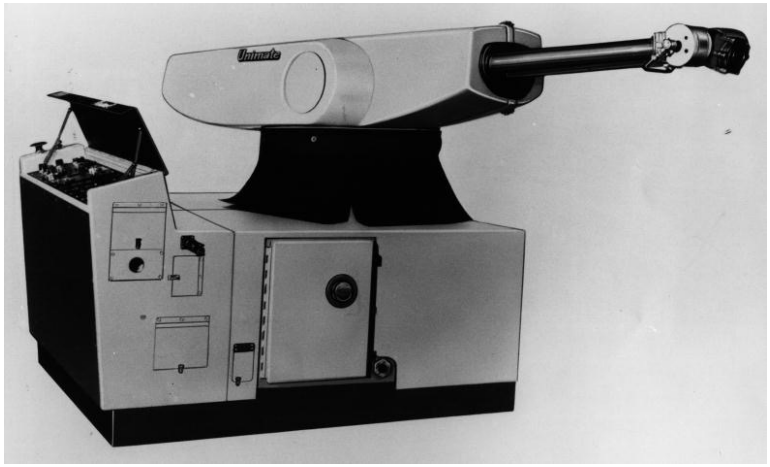
LUND UNIVERSITY

The Industrial Robot

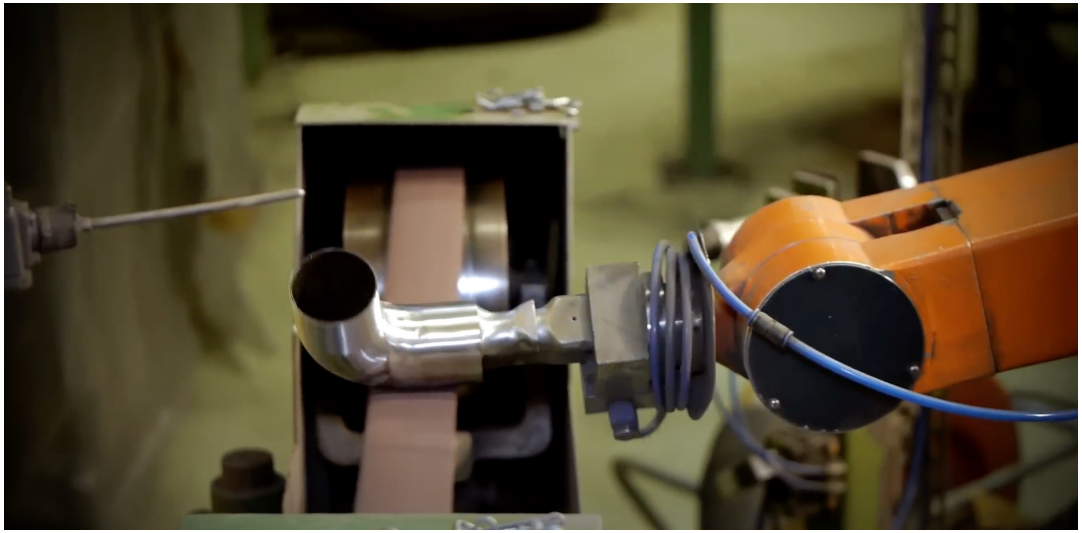


Patent of first robot by Cyril W. Kenwards (1957)

Unimate 2000, first generation of Industrial robot (1961).
Based on a patent by George C. Devol



The First Completely Electrically Driven and Microprocessor Controlled Robot (1973)



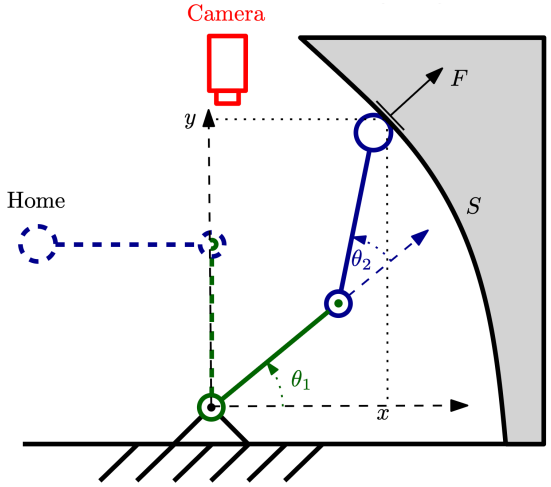
https://www.youtube.com/watch?v=2xNgQhLAPyI

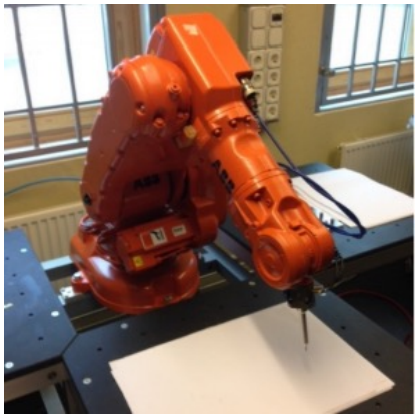
Robotics in This Course

- The following conceptual problems must be resolved to make a robot succeed in performing a typical task:
  - **Forward Kinematics**
  - Inverse Kinematics
  - Velocity Kinematics / Jacobians
  - Dynamics
  - Path Planning and Trajectory Generation
  - Motion Control
  - (Force Control)
  - Sequence programming (and task description)

LUND UNIVERSITY

From Task Description to Performed Task

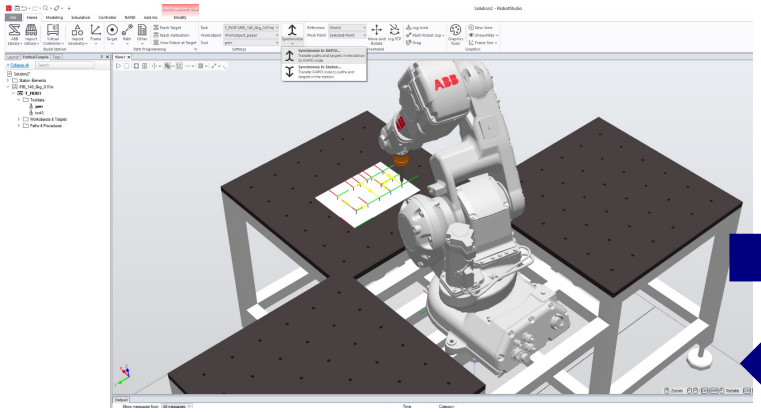


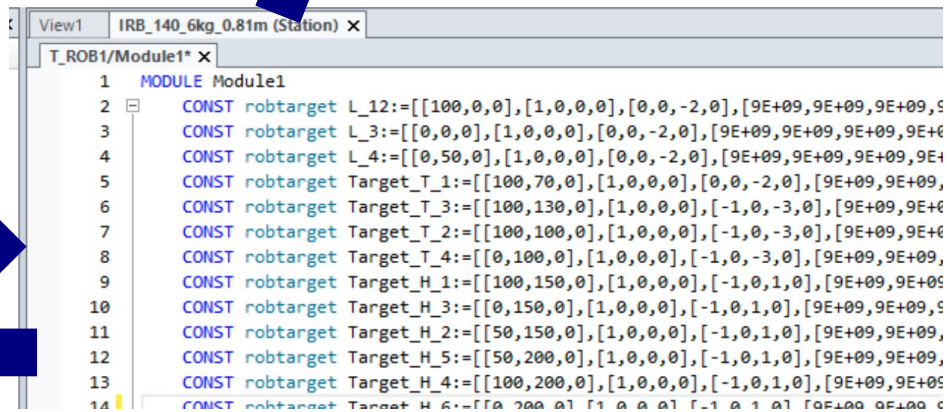


Applied **robot programming**: Robot + CAD o[illegible]workcell

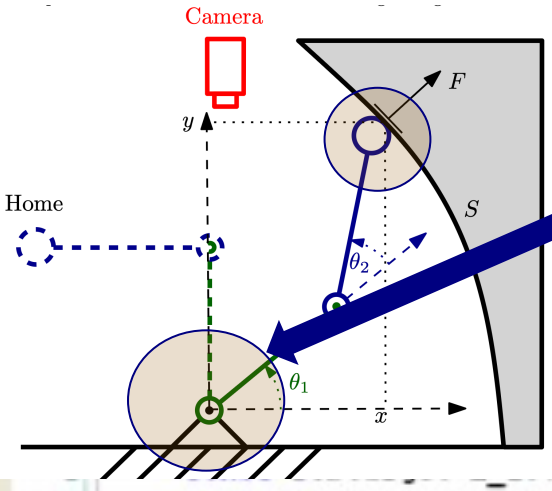
RobotStudio Exercises

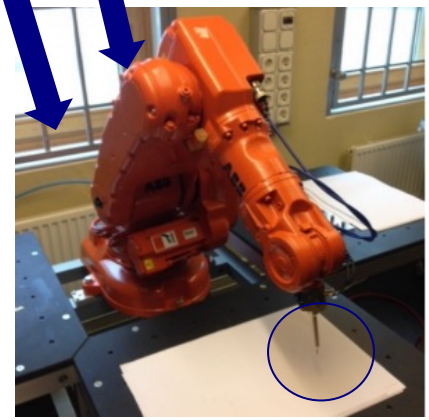
(1) Paths, frames and configurations (2) I/O (3) [illegible]llision avoidance





So How is it Working Behind the Scenes? – From Program Instruction to Motor Angles





```
4     CONST robtarget L_20:=[[100,0,0],[1,0,0,0],[0,0,2,0],[9E9,9E9,9
5     CONST robtarget L_30:=[[0,50,0],[1,0,0,0],[0,0,2,0],[9E9,9E9,9E
6   PROC Path_L()
7     MoveL Target_10,v1000,z100,pen\WObj:=wobj0;
8     MoveL L_20,v1000,z5,pen\WObj:=Workobject_1;
9     MoveL Offs(L_20,0,0,15),v100,z1,pen\WObj:=Workobject_1;
10    MoveL Offs(L_10,0,0,15),v100,z1,pen\WObj:=Workobject_1;
```

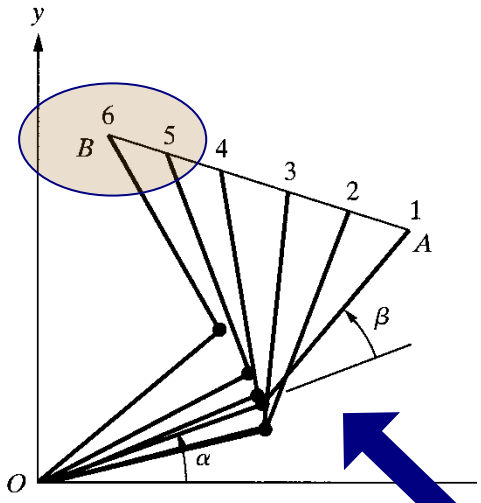
From MoveL L_20 to joint values [q1,…,q6]

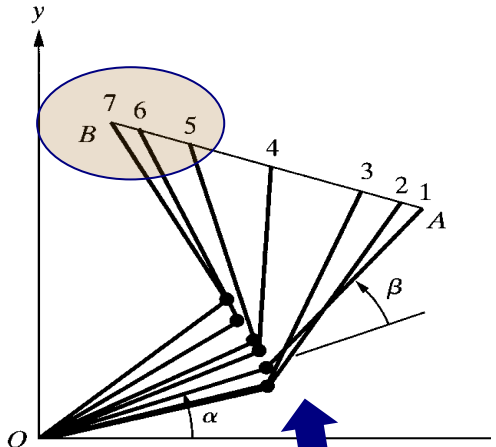
(Lec 2, 3) Frames, Forward/Inverse kinematics (joint angles ←→ pose)

(Lec 3) Relation between velocities (Jacobian)

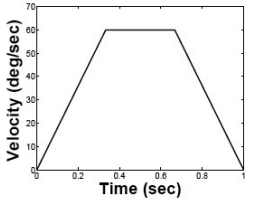
Computer exercises [Matlab / Python]: frames, DH-modeling,…

From Program Instruction to Paths and Trajectories of Motor Angles

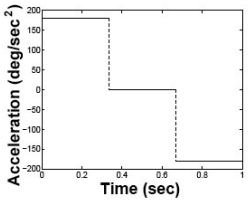




Velocity and acceleration constraints in joint space or in Cartesian motion (or combination)



(b)



(c)

```
4     CONST rob  rget L_20:=[[100,0,0],[1,0,0,0],[0  ,2,0],[9E9,9E9,9
5     CONST robt  et L_30:=[[0,50,0],[1,0,0,0],[0,  2,0],[9E9,9E9,9E
6   PROC Path_L()
7     MoveL Target_10  1000,z100,pen\WObj:=wobj0;
8     MoveL L_20,v1000,  pen\WObj:=Workobject_1;
9     MoveL Offs(L_20,0,0  ),v100,z1,pen\WObj:=Worko  ject_1;
10    MoveL Offs(L_10,0,0,1  v100,z1,pen\WObj:=Worko  ect_1;
```

From MoveL L_20 to joint values [q1,…,q6]

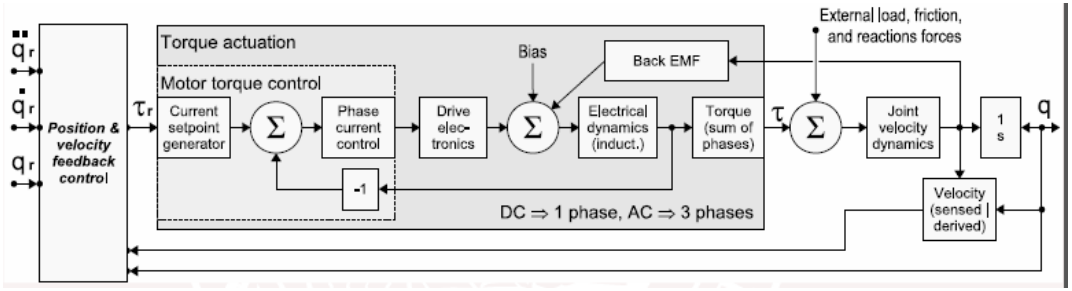
(Lec 2, 3) Frames, Forward/Inverse kinematics (joint angles ←→ pose)

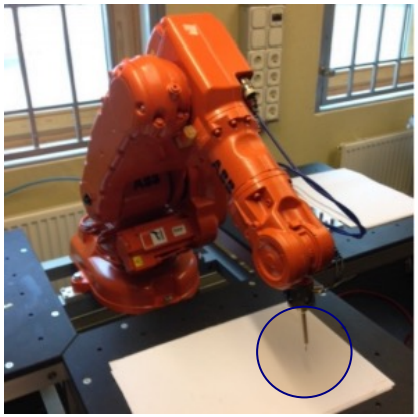
(Lec 3) Relation between velocities (Jacobian)

(Lec 4) Paths and trajectories (geometric vs. dynamic due to v_max, tau_max)

Computer exercises [Matlab / Python]: frames, DH-modeling, trajectory generation

From Program Instruction to Motor Angles





```
4     CONST robtarget L_20:=[100,0,0],[1,0,0,0],[0,0,2,0],[9E9,9E9,9
5     CONST robtarget L_30:=[0,50,0],[1,0,0,0],[0,0,2,0],[9E9,9E9,9E
6   PROC Path_L()
7     MoveL Target_10,v1000,z10,pen\WObj:=wobj0;
8     MoveL L_20,v1000,z5,pen\WObj:=Workobject_1;
9     MoveL Offs(L_20,0,0,15),v100,z1,pen\WObj:=Workobject_1;
10    MoveL Offs(L_10,0,0,15),v100,z1,pen\WObj:=Workobject_1;
```

From MoveL L_20 to joint values [q1,…,q6]

(Lec 2, 3) Frames, Forward/Inverse kinematics (joint angles ←→ pose)

(Lec 3) Relation between velocities (Jacobian)

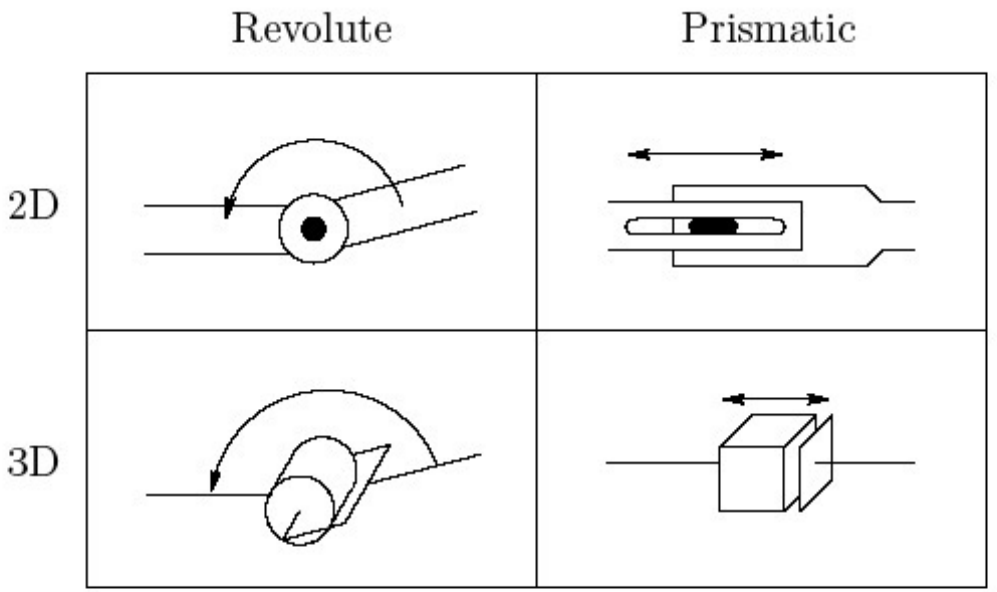
(Lec 5–6) ”From q_{ref} to q” , servo control: PID + FeedForward

(Lec 8–9) Robot modeling to find dynamic process model

Transforms and Forward Kinematics

Mathematical Modeling of Robots

- Links and joints



- Configuration space

A ***configuration*** of a manipulator is a complete specification of the location of every point on the manipulator.
The set of all configurations is called the ***configuration space***.

Degrees of Freedom (Recap)

- An object has n degrees of freedom (DOF) if its configuration can be minimally specified by n parameters.
- The number of DOF is equal to the dimension of the configuration space.
- For a robot manipulator:
  number of joints = number of DOF

ABB YuMi

Articulated Manipulator: ABB IRB1400

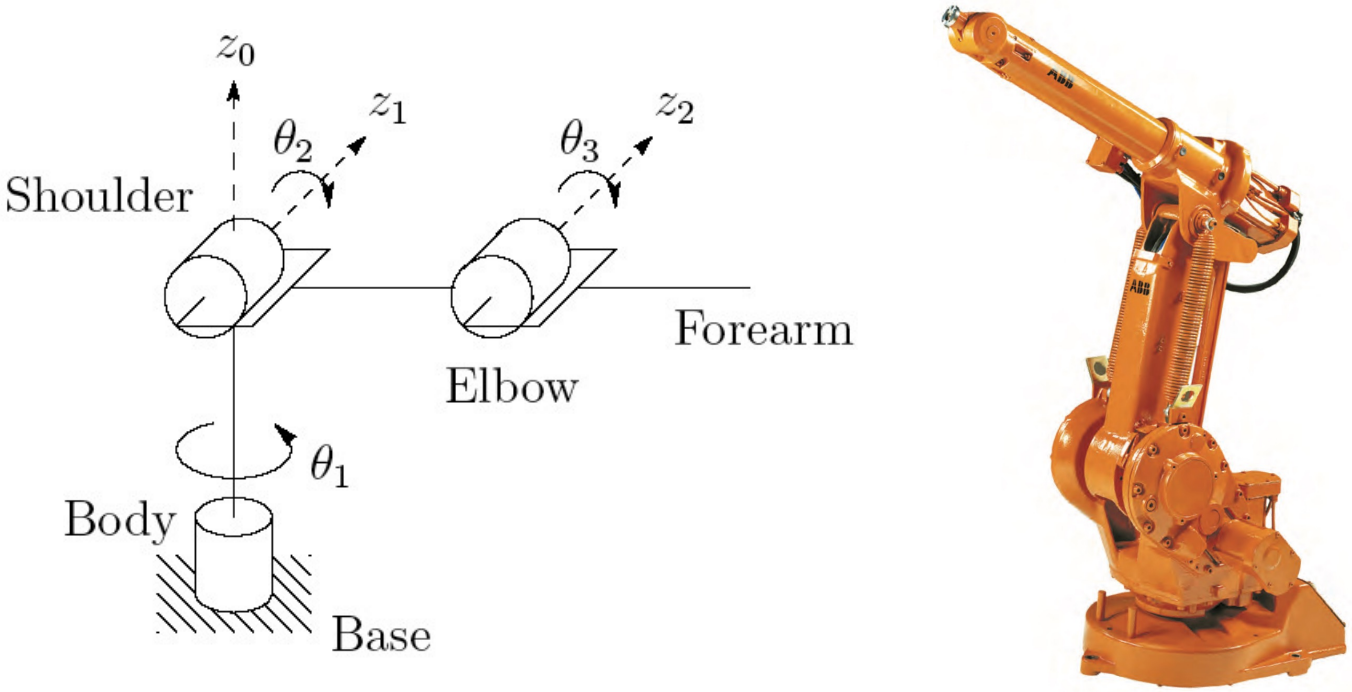


Figure 1.9: The ABB IRB1400 Robot, a six-DOF elbow manipulator (right). The symbolic representation of this manipulator (left) shows why it is referred to as an anthropomorphic robot. The links and joints are analogous to human joints and limbs. (Photo courtesy of ABB.)

SCARA Manipulator: Adept Cobra Smart600

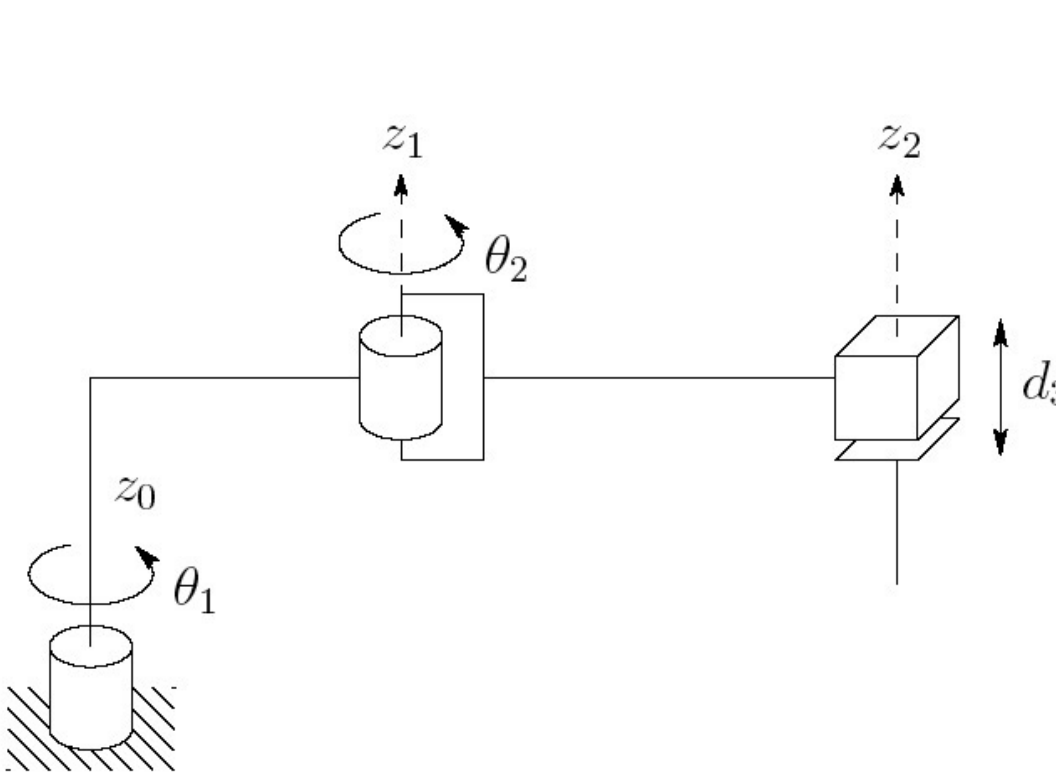


Figure 1.13: Symbolic representation of the SCARA arm.

Cylindrical Robot: Seiko RT3300

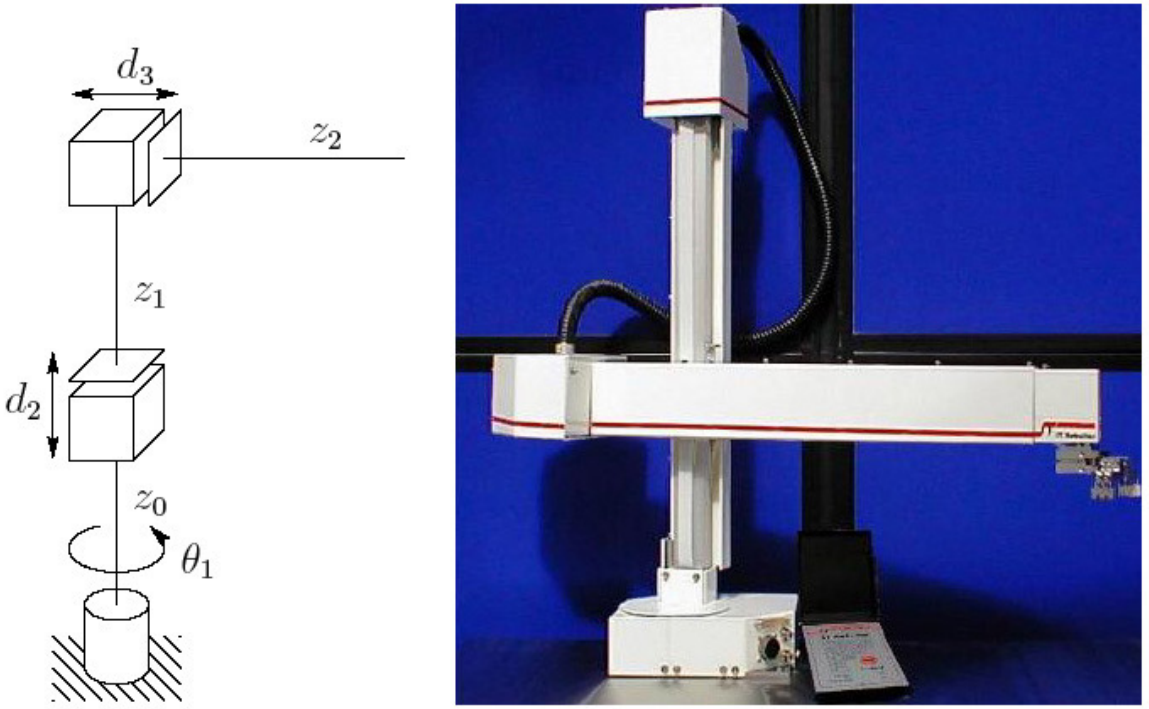
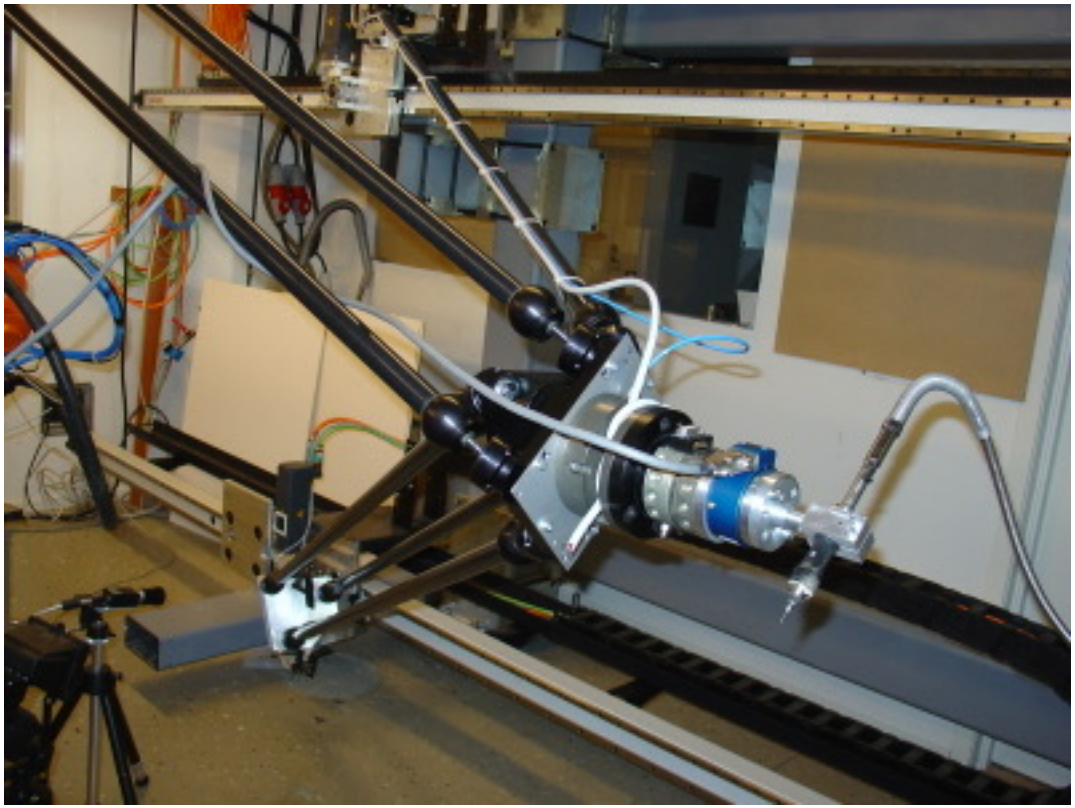


Figure 1.15: The Seiko RT3300 Robot cylindrical robot. Cylindrical robots are often used in materials transfer tasks. (Photo courtesy of Epson Robots.)

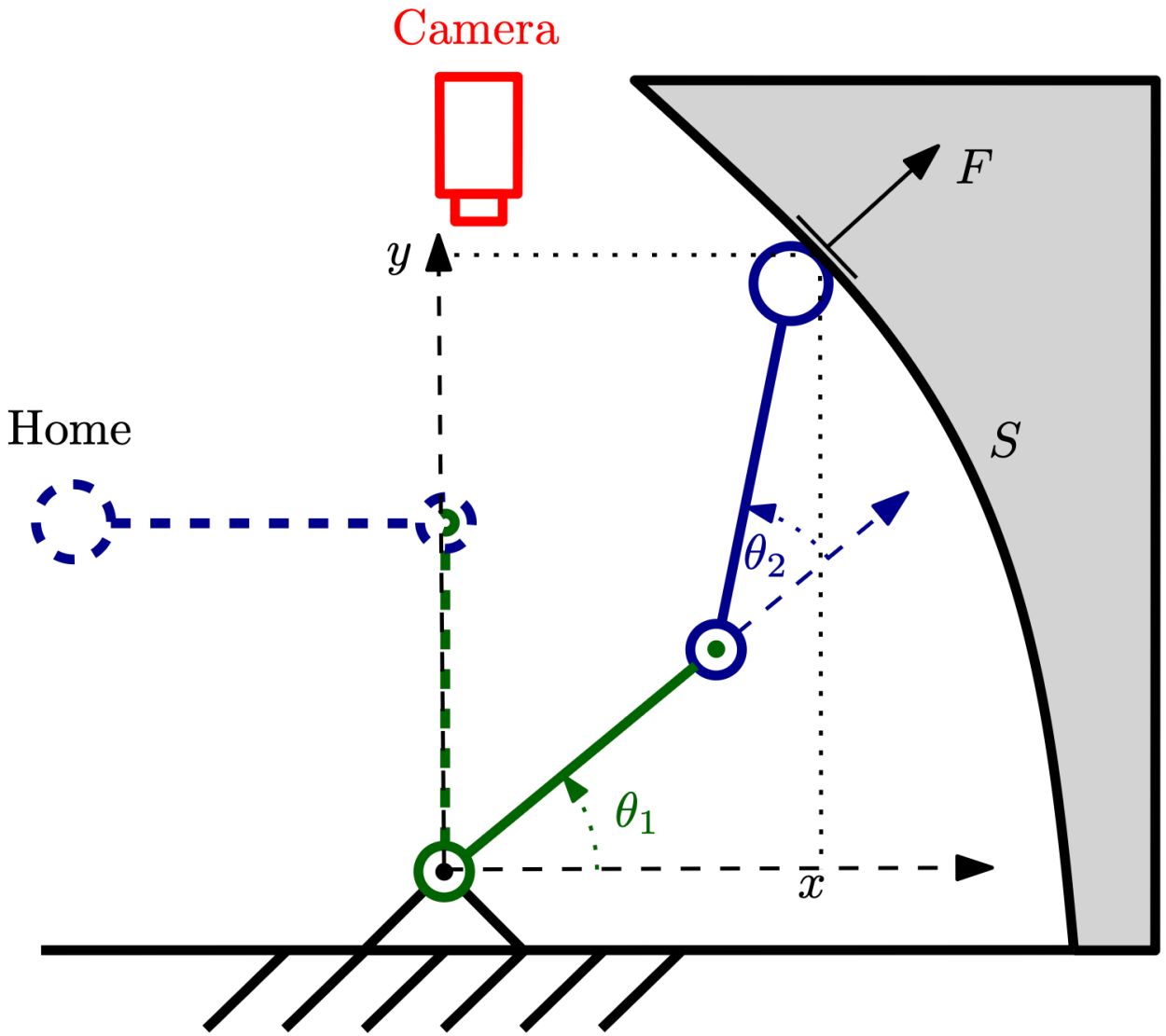
Gantry-Tau (Linear Actuators for PKM)



[Discuss 2 min]

https://www.youtube.com/watch?v=HKe1FXP_ajg

Robot Task Example



Forward Kinematics

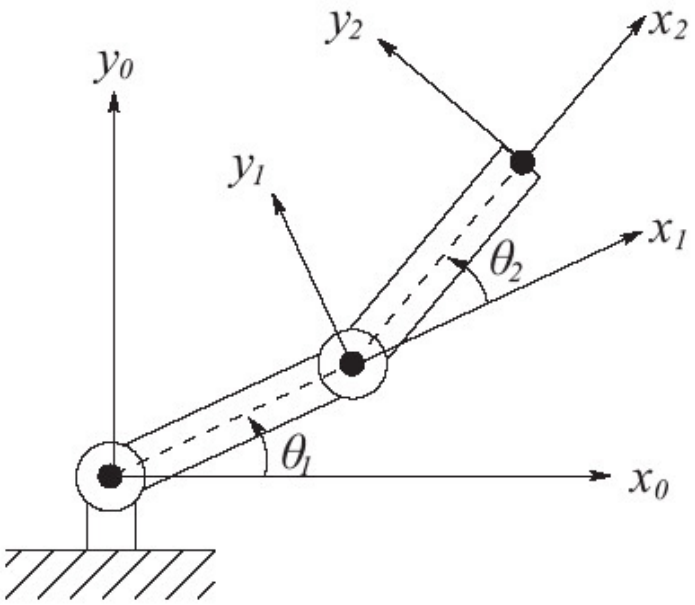


Figure 1.20: Coordinate frames attached to the links of a two-link planar robot. Each coordinate frame moves as the corresponding link moves. The mathematical description of the robot motion is thus reduced to a mathematical description of moving coordinate frames.

Inverse Kinematics

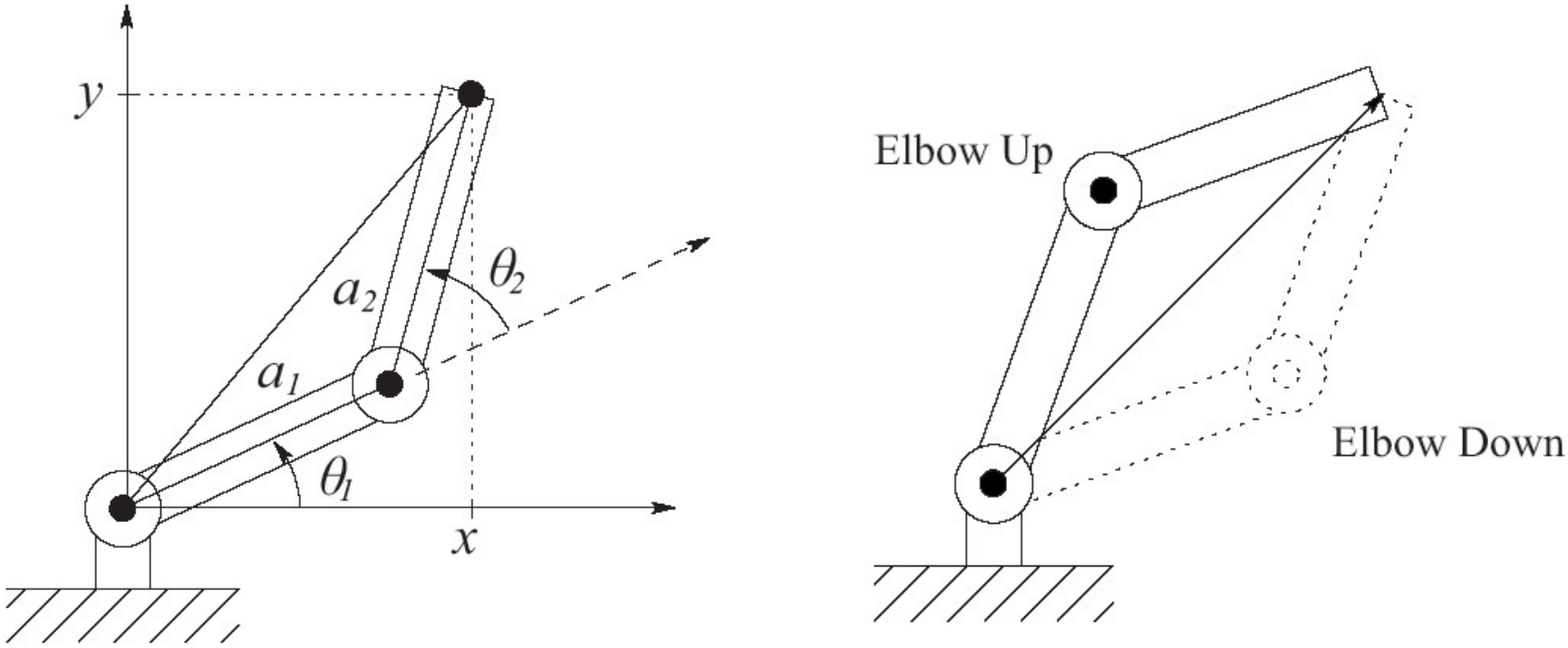
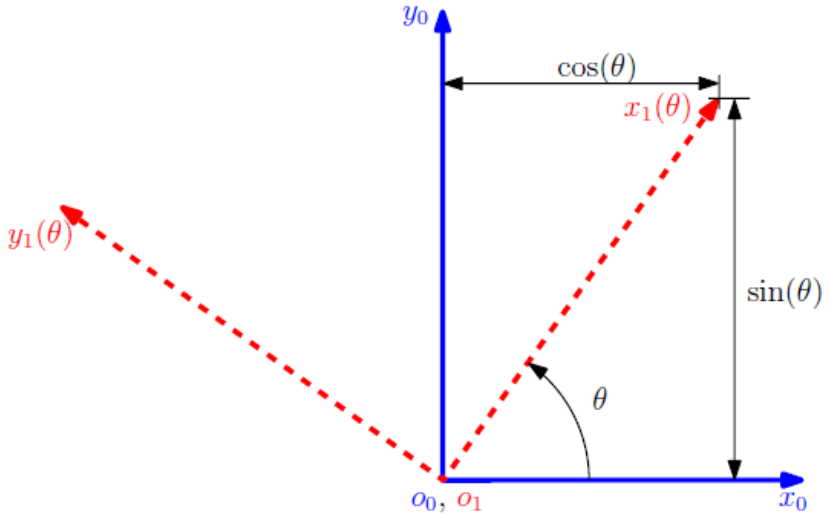
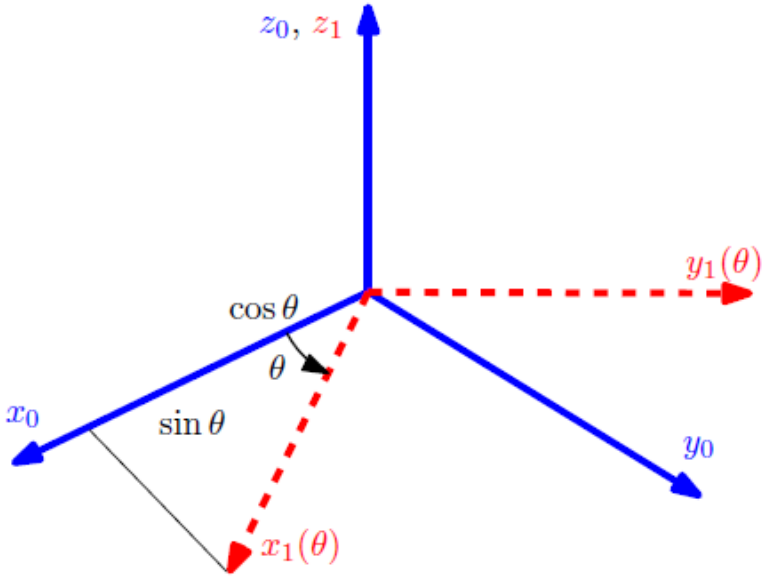


Figure 1.21: The two-link elbow robot has two solutions to the inverse kinematics except at singular configurations, the elbow up solution and the elbow down solution.

Rotations and Translations in 2D/3D

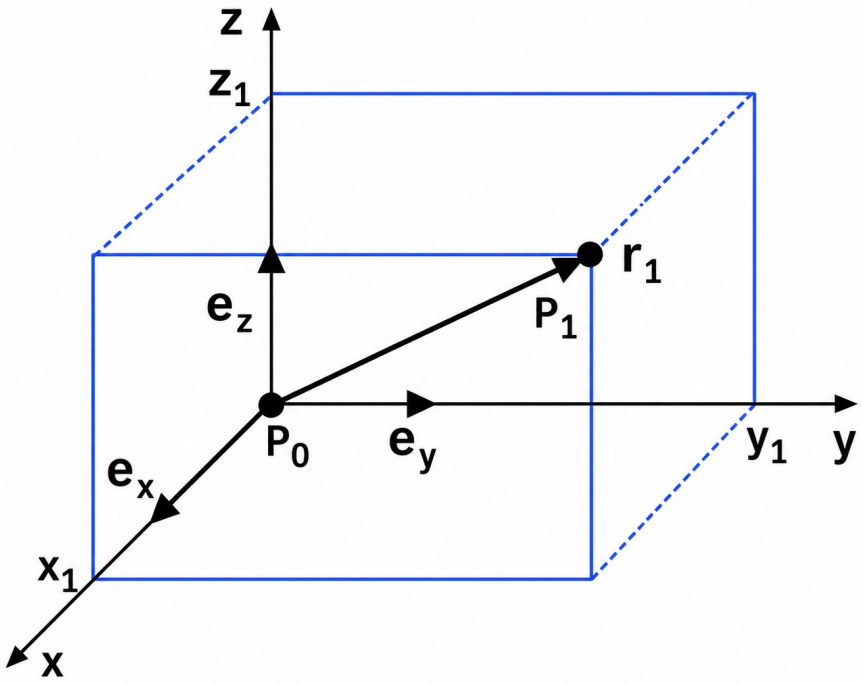


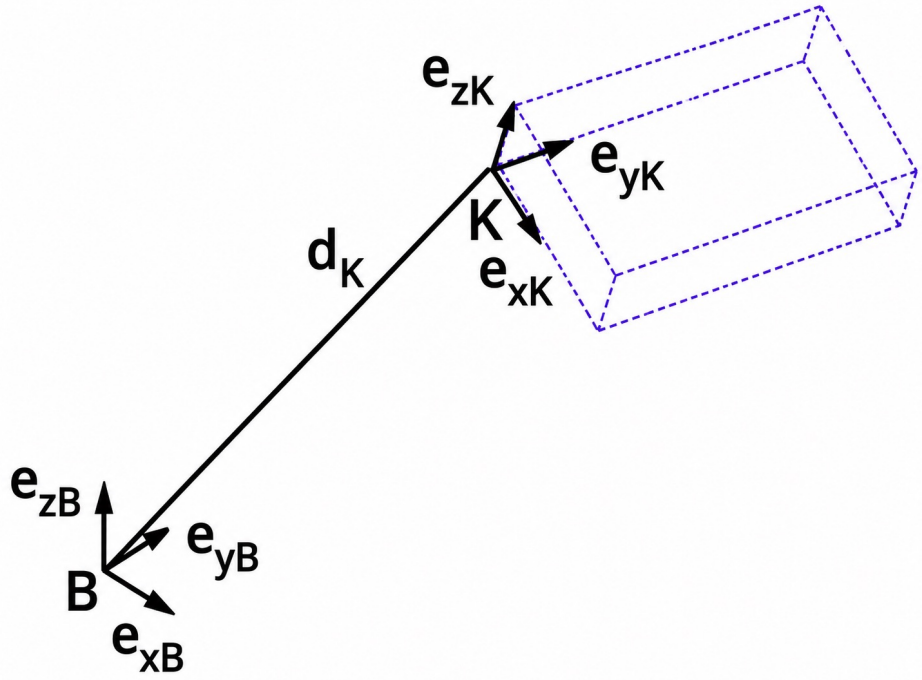


$$R_1^0(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}$$

$$R_1^0(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad \{=: R_{z,\theta}\}$$

Representing Positions & Orientations





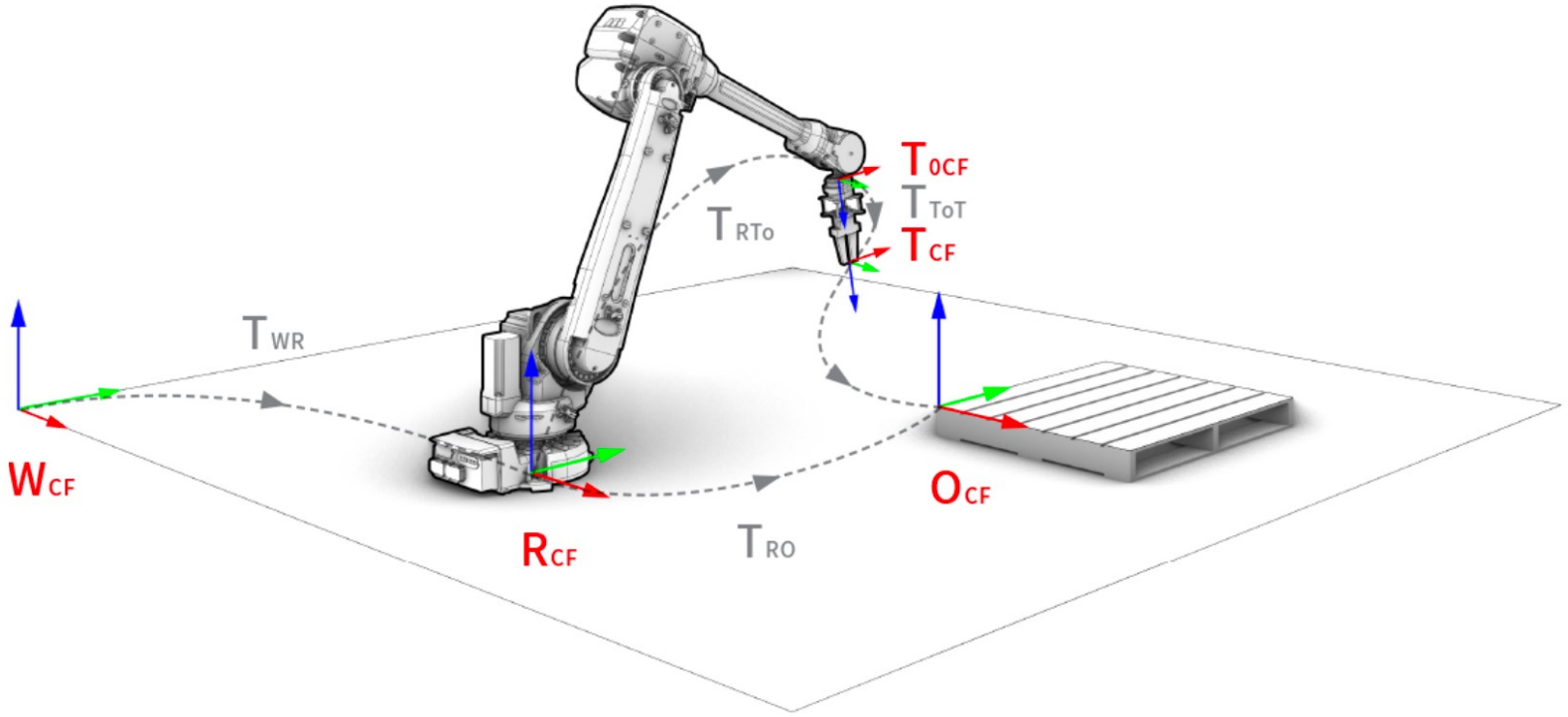
LUND UNIVERSITY

Conversions Between 3D Pose Representations

	Output type										
Input type	t	Euler	RPY	θ, v	R	T	Twist vector	Twist	Unit-Quaternion	SO3	SE3
t (3-vector)						`transl`		`Twist('T')`			`SE3()`
Euler (3-vector)					`eul2r`	`eul2tr`			`UnitQuater-nion.eul()`	`SO3.eul()`	`SE3.eul()`
RPY (3-vector)					`rpy2r`	`rpy2tr`			`UnitQuater-nion.rpy()`	`SO3.rpy()`	`SE3.rpy()`
θ, v (scalar + 3-vector)					`angvec2r`	`angvec2tr`			`UnitQuater-nion.angvec()`	`SO3.angvec()`	`SE3.angvec()`
R (3×3 matrix)		`tr2eul`	`tr2rpy`	`tr2angvec`		`r2t`	`trlog`		`UnitQuater-nion()`	`SO3()`	`SE3()`
T (4×4 matrix)	`transl`	`tr2eul`	`tr2rpy`	`tr2angvec`	`t2r`		`trlog`	`Twist()`	`UnitQuater-nion()`	`SO3()`	`SE3()`
Twist vector (3- or 6-vector)					`trexp`	`trexp`		`Twist()`		`SO3.exp()`	`SE3.exp()`
Twist						`.T`	`.S`				`.SE`
Unit-Quaternion		`.toeul`	`.torpy`	`.toangvec`	`.R`	`.T`				`.SO3`	`.SE3`
SO3		`.toeul`	`.torpy`	`.toangvec`	`.R`	`.T`	`.log`		`.UnitQuater-nion`		`.SE3`
SE3	`.t`	`.toeul`	`.torpy`	`.toangvec`	`.R`	`.T`	`.log`	`.Twist`	`.UnitQuater-nion`	`.SO3`	

Overview of pose representations from Peter Corke's Robotics Toolbox.

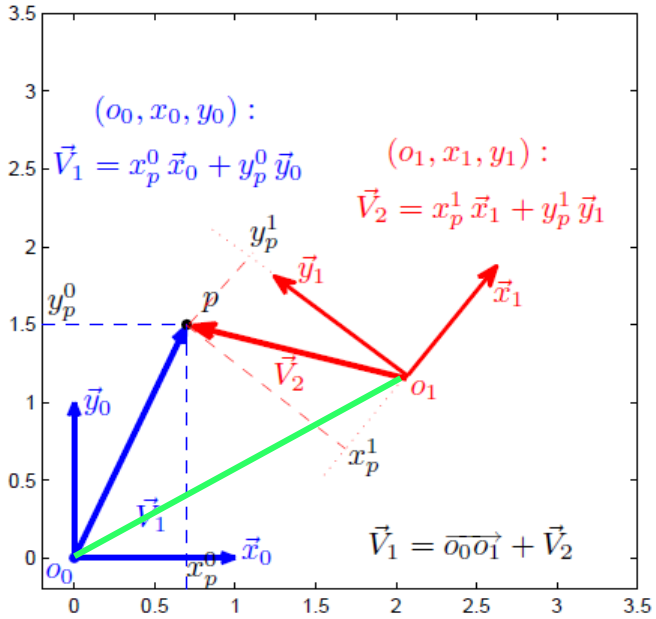
Coordinate Frames in Robotics



From compas_fab_documentation

Homogeneous Transformation

Combine both rotation and translation in one 4x4 matrix



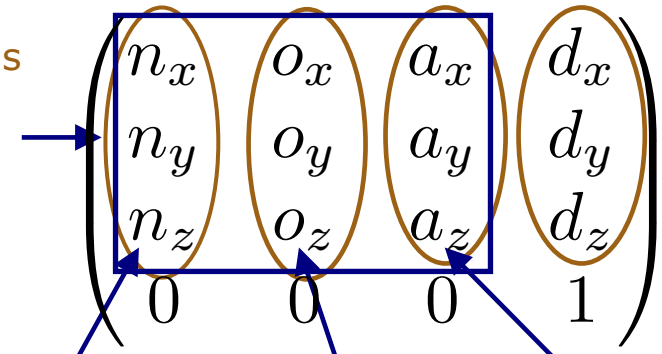
$$\underbrace{\begin{bmatrix} p^0 \\ 1 \end{bmatrix}}_{P^0} = \begin{bmatrix} x_p^0 \\ y_p^0 \\ z_p^0 \\ 1 \end{bmatrix} = \begin{bmatrix} (x_1^0)_x & (y_1^0)_x & (z_1^0)_x & (o_1^0)_x \\ (x_1^0)_y & (y_1^0)_y & (z_1^0)_y & (o_1^0)_y \\ (x_1^0)_z & (y_1^0)_z & (z_1^0)_z & (o_1^0)_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_p^1 \\ y_p^1 \\ z_p^1 \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} R_1^0 & o_1^0 \\ 0_{1\times 3} & 1 \end{bmatrix}}_{H_1^0} \underbrace{\begin{bmatrix} p^1 \\ 1 \end{bmatrix}}_{P^1}$$

Homogeneous Transformation (cont'd)

A 4x4 matrix that describes "3-space" with information that relates orientation and position (pose) of a remote space to a local space

This 3x3 'sub-matrix' is the information that relates orientation of $\text{Frame}_{\text{rem}}$ to $\text{Frame}_{\text{local}}$

(This is called *R*, the rotational submatrix)



D vector is the position of the *origin* of the remote space in local coordinate dimensions

N vector projects the X_{rem} axis to the local coordinate system

O vector projects the Y_{rem} axis to the local coordinate system

A vector projects the Z_{rem} axis to the local coordinate system

LUND UNIVERSITY

Inverse Homogeneous Transformation

Changing our ”point of view”:

Going in the “other direction”

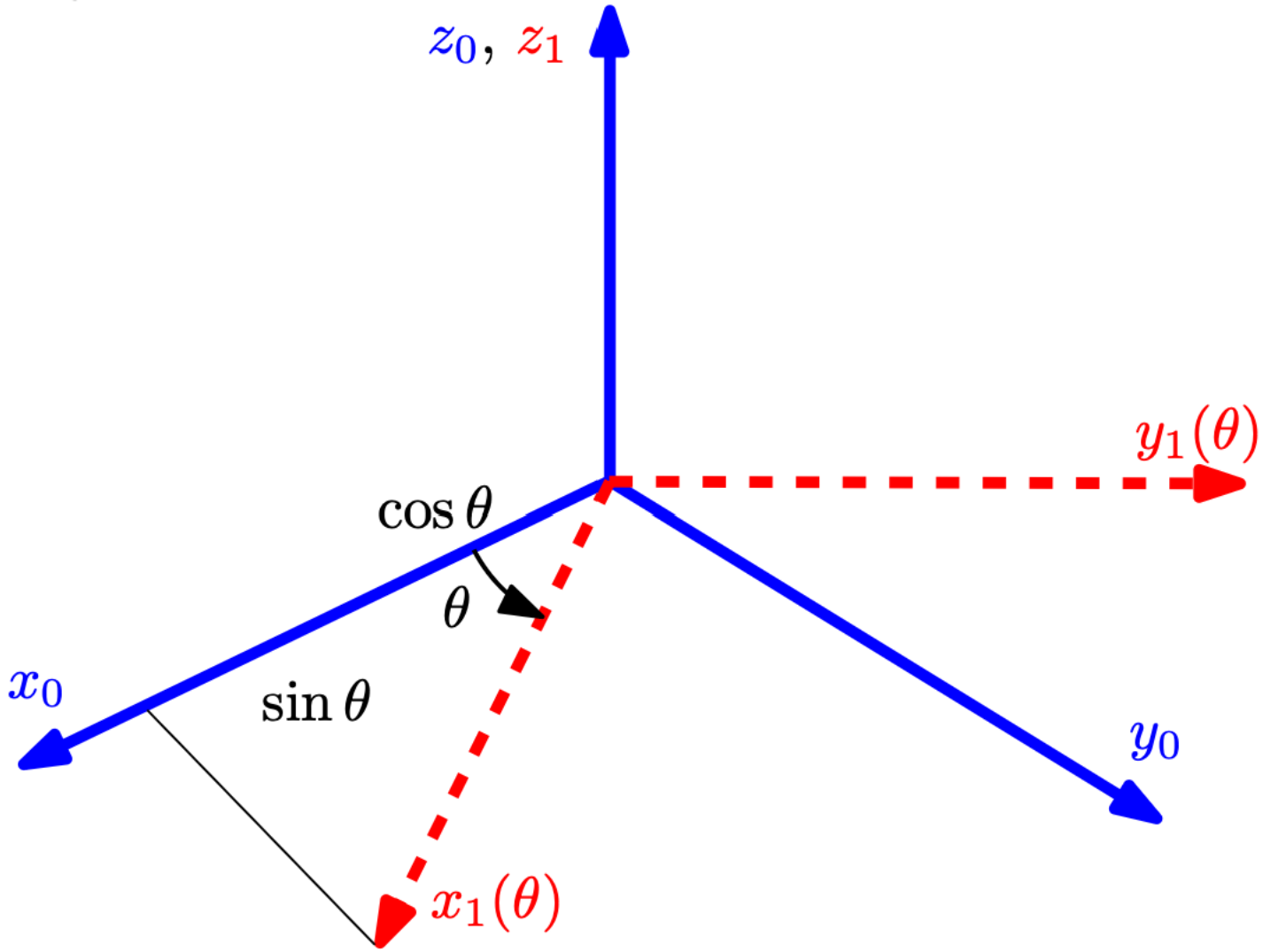
⟶ Inverse of transformation matrix

$$\begin{pmatrix} n_x & n_y & n_z & -(\vec{n}\cdot\vec{d}) \\ o_x & o_y & o_z & -(\vec{o}\cdot\vec{d}) \\ a_x & a_y & a_z & -(\vec{a}\cdot\vec{d}) \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

This is the transpose of the *R* sub-matrix of the original HTM

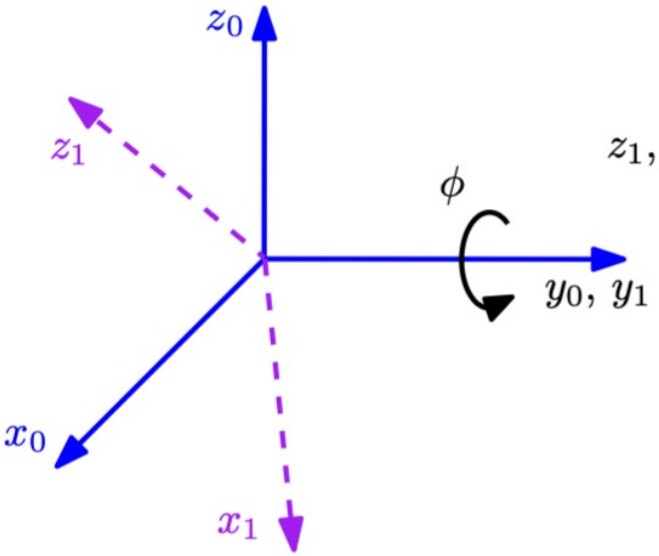
LUND UNIVERSITY

Rotations About the Primary Axes

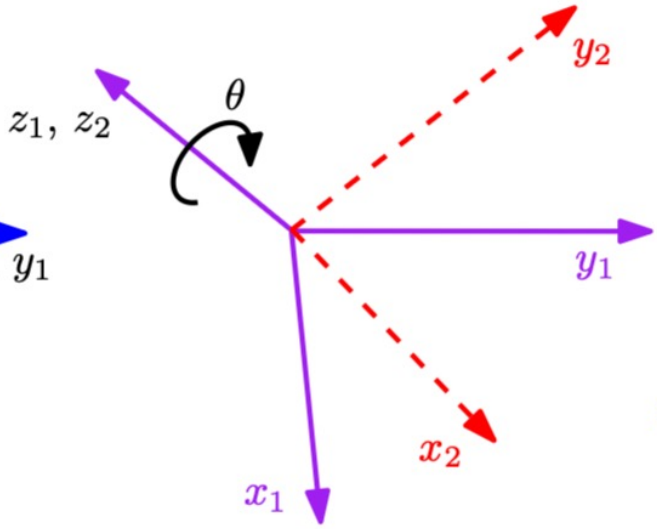


Rotations About Current Axes

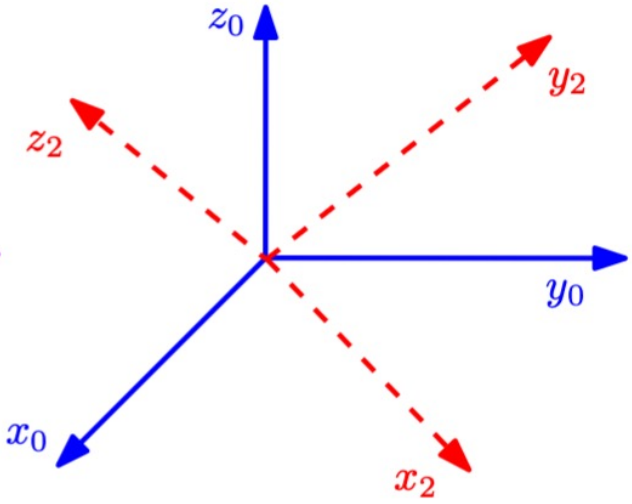
Around the y-axis (y_0)



Around the z-axis (z_1)

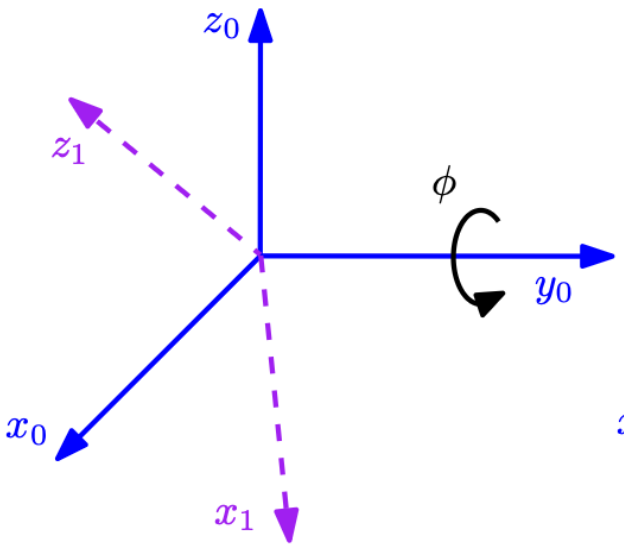


Overall

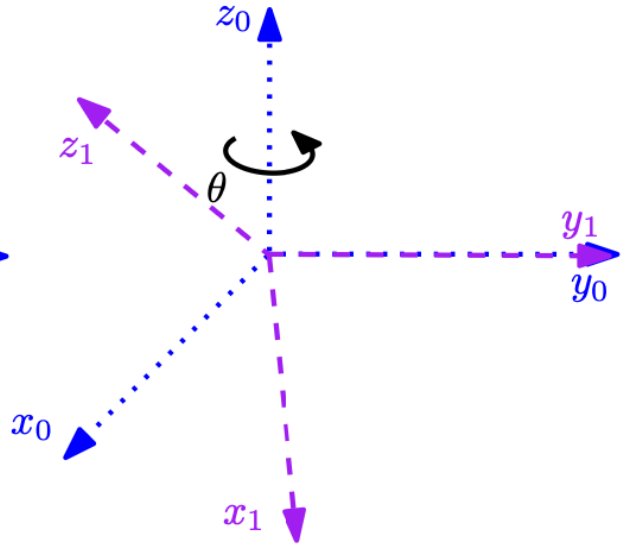


Rotations with Respect to Fixed Frame

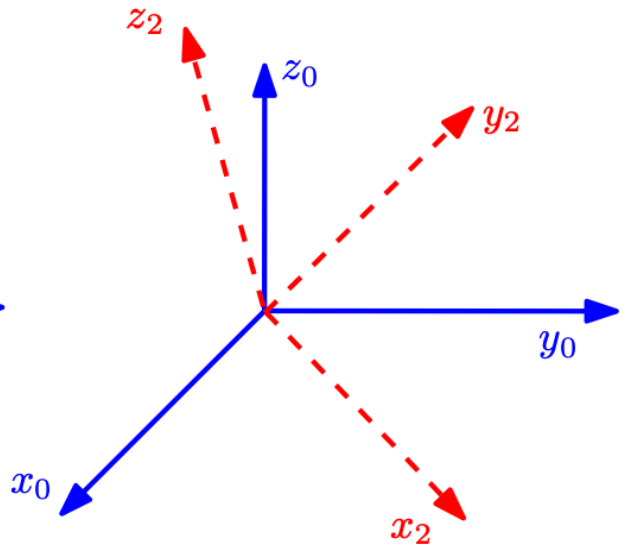
Around the y-axis (y_0)

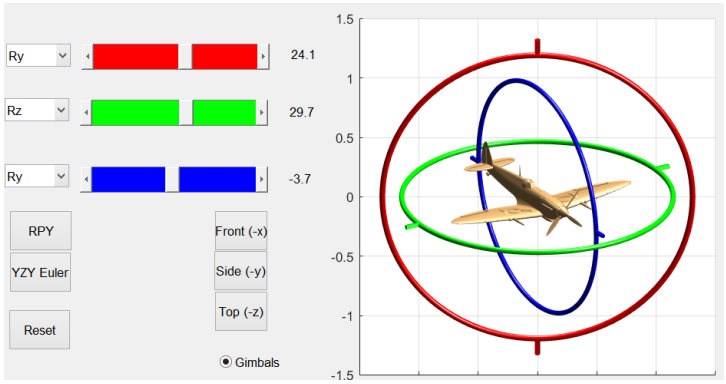


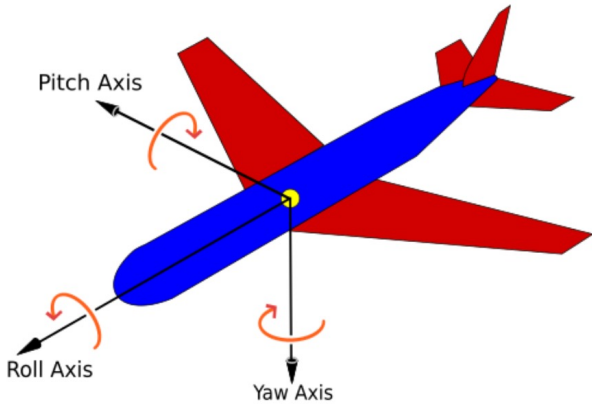
Around the z-axis (z_0)



Overall

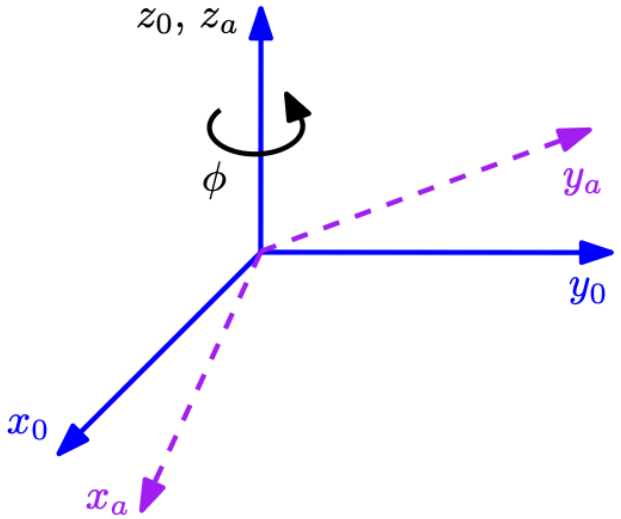




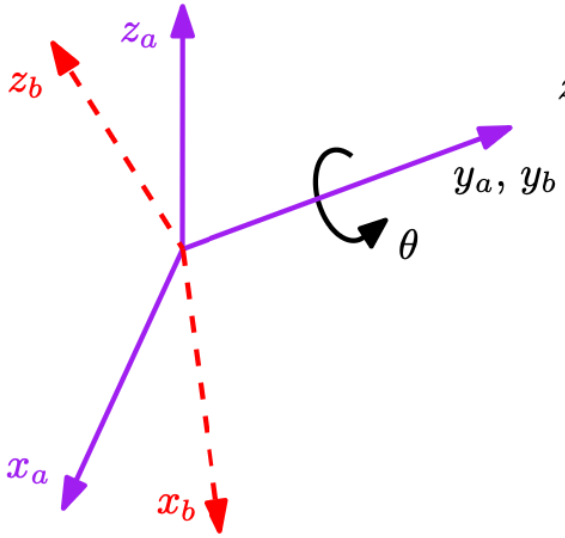


Euler Angles (z_0 y_a z_b)

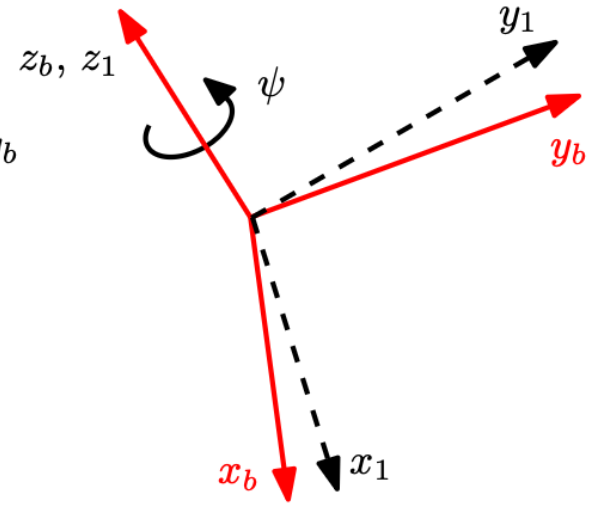
Around the z-axis (z_0 by ϕ)



Around the y-axis (y_a by θ)



Around the z-axis (z_b by ψ)



$$(x_0, y_0, z_0) \xrightarrow{\phi} (x_a, y_a, z_a) \xrightarrow{\theta} (x_b, y_b, z_b) \xrightarrow{\psi} (x_1, y_1, z_1)$$

Euler Angles (cont'd)

$$R_{zyz}(\varphi,\theta,\psi) = R_z(\varphi)R_y(\theta)R_z(\psi)$$

$$R_{zyz}(\varphi,\theta,\psi) = \begin{pmatrix} c_\varphi & -s_\varphi & 0 \\ s_\varphi & c_\varphi & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{pmatrix} \begin{pmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$R_{zyz}(\varphi,\theta,\psi) = \begin{pmatrix} c_\varphi c_\theta c_\psi - s_\varphi s_\psi & -c_\varphi c_\theta s_\psi - s_\varphi c_\psi & c_\varphi s_\theta \\ s_\varphi c_\theta c_\psi + c_\varphi s_\psi & -s_\varphi c_\theta s_\psi + c_\varphi c_\psi & s_\varphi s_\theta \\ -s_\theta c_\psi & s_\theta s_\psi & c_\theta \end{pmatrix}$$

LUND UNIVERSITY

Example

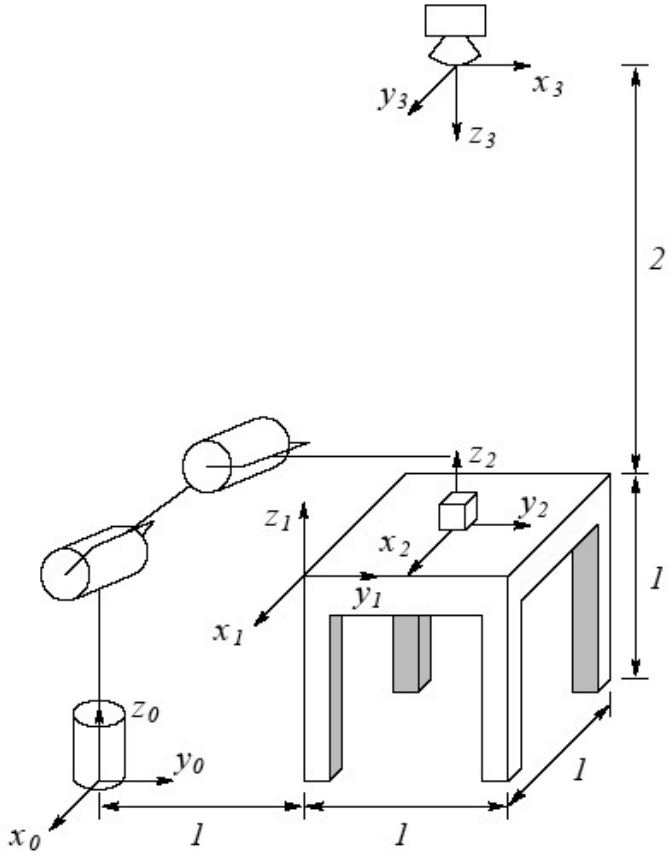


Figure 2.14: Diagram for Problem 2-39.

- A cube measuring 200 mm on a side is placed at the center of the table.
- Frame 2 should be in the center of the cube, i.e., slightly above the table.
- A camera is situated directly above the center of the cube.

1. Find the homogeneous transformations relating Frames 1, 2 and 3 to the base frame 0.

2. Find the homogeneous transformations relating Frame 3 to Frame 2.

[Discuss 2 min]

Example – Solution

$$H_1^0 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \qquad H_2^0 = \begin{pmatrix} 1 & 0 & 0 & -0.5 \\ 0 & 1 & 0 & 1.5 \\ 0 & 0 & 1 & 1.1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \qquad H_3^0 = \begin{pmatrix} 0 & 1 & 0 & -0.5 \\ 1 & 0 & 0 & 1.5 \\ 0 & 0 & -1 & 3 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$H_2^0 = H_1^0 H_2^1$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -0.5 \\ 0 & 1 & 0 & 0.5 \\ 0 & 0 & 1 & 0.1 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & -0.5 \\ 0 & 1 & 0 & 1.5 \\ 0 & 0 & 1 & 1.1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$H_3^2 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 1.9 \\ 0 & 0 & 0 & 1 \end{pmatrix} \qquad H_2^3 \text{ inverse of } H_3^2$$

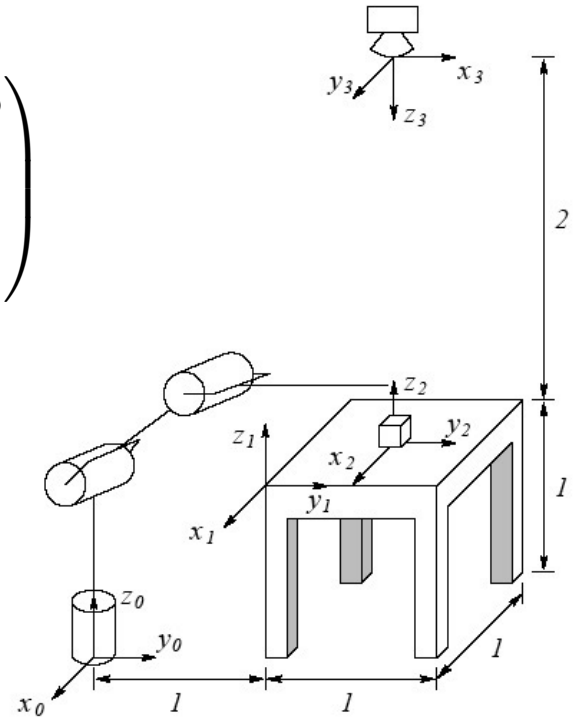


Figure 2.14: Diagram for Problem 2-39.

Composing Transformations

Rules for *Composition of Rotational Transformations*:

- Rotation about current axes: Postmultiply matrix
- Rotation relative to the fixed frame: Premultiply matrix

- Study details and examples in Chapter 2 of the lecture notes!

Quaternions

Euler's rotation theorem: *Any rotation in three dimensions can be represented as a combination of an axis vector and an angle of rotation about that axis.*

Quaternions give a simple way to

- encode this axis-angle representation in four numbers
- apply the corresponding rotation to a position vector representing a point relative to the origin in R^3

Describing rotations with quaternions

Let (q_0, q_1, q_2, q_3) be the coordinates of a rotation by α about the axis defined by a unit vector $\vec{u}$ (axis of rotation).

A quaternion is defined by

$$q = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k} = q_0 + (q_1, q_2, q_3) = \cos(\alpha/2) + \vec{u}\sin(\alpha/2)$$

LUND UNIVERSITY

Annual Walk on ”Broomsday” (October 16)

“Here as he walked by
on the 16th of October 1843
Sir William Rowan Hamilton
in a flash of genius discovered
the fundamental formula for
quaternion multiplication

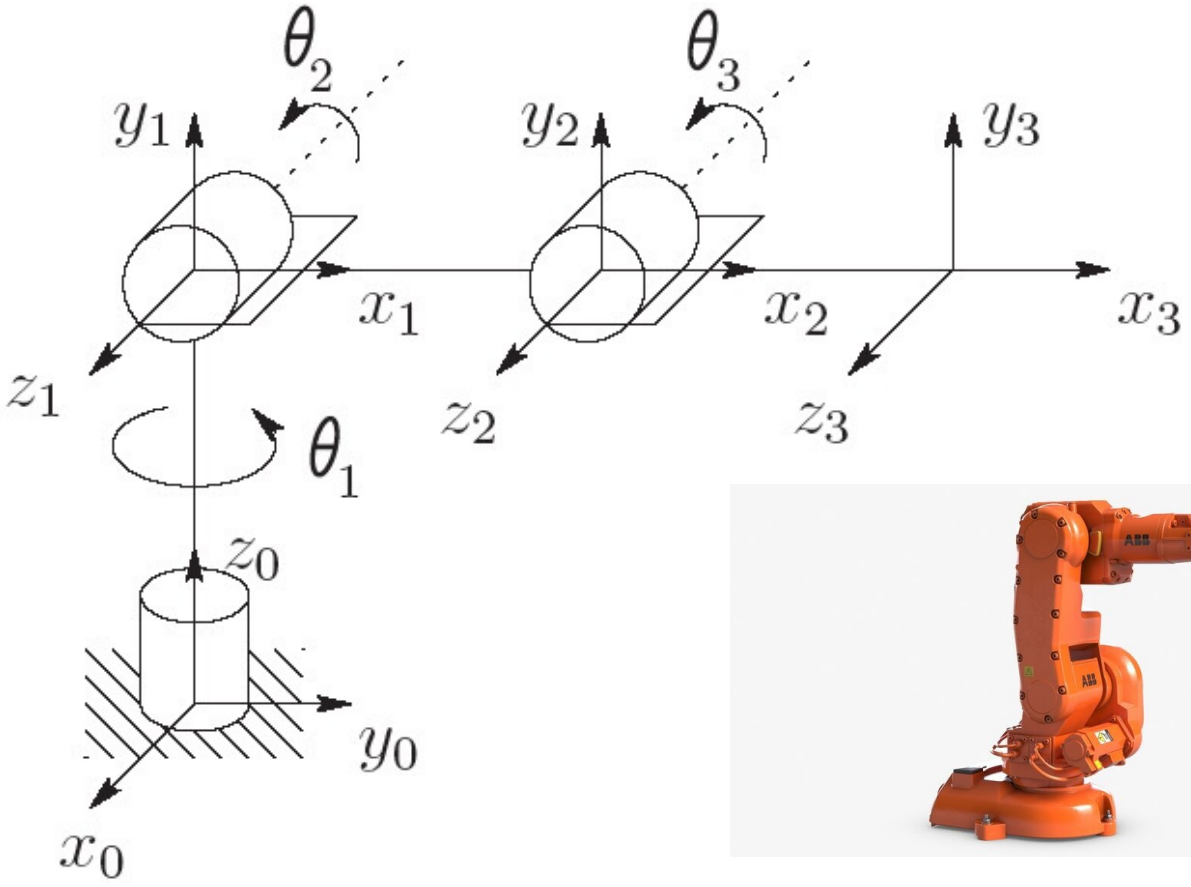
$$i^2 = j^2 = k^2 = ijk = -1$$

& cut it on a stone of this bridge”.

[Source: https://en.wikipedia.org/wiki/Broom_Bridge]

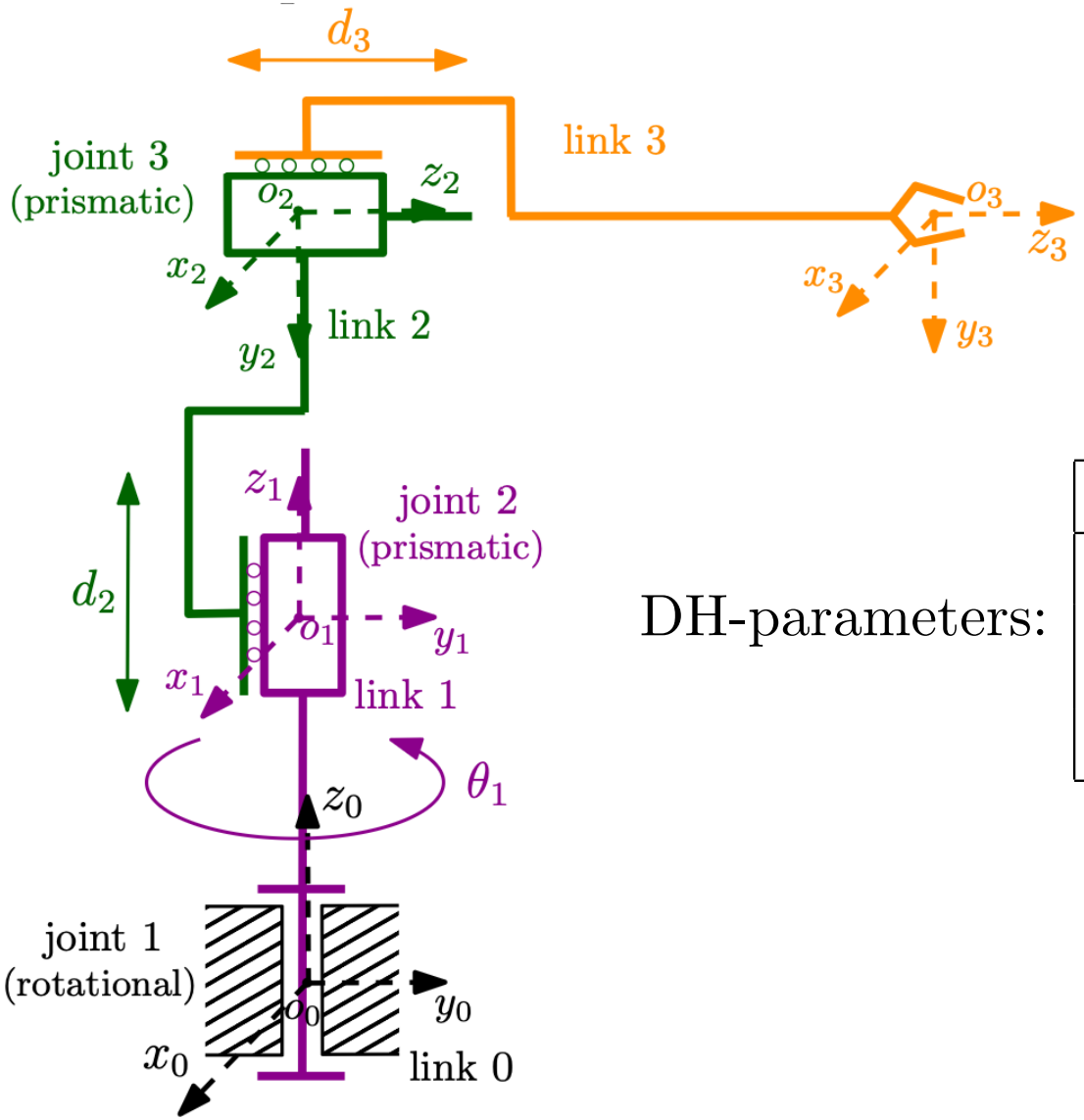
Forward Kinematics

Find the position and orientation of the end effector / tool given the values for the joint variables



LUND UNIVERSITY

Representation from Link-to-Link (Outwards)

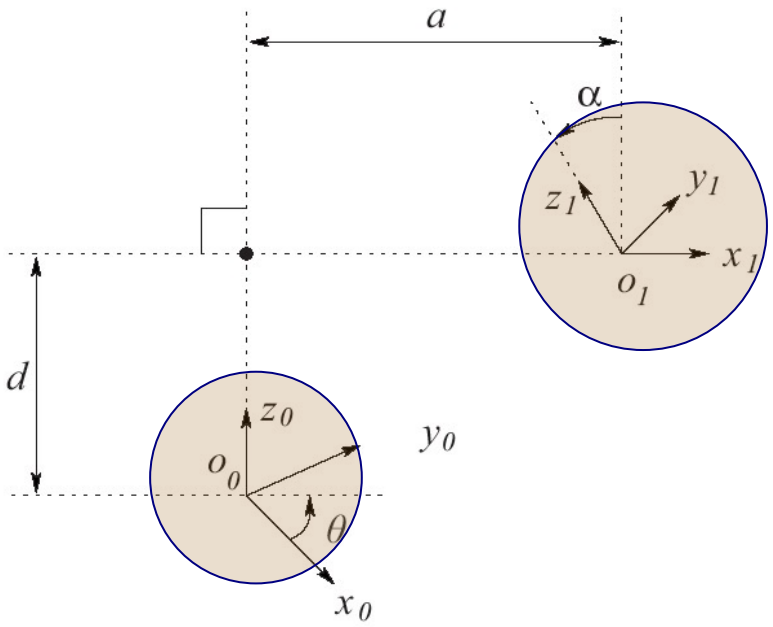


DH-parameters:

angle, link offset, link length, and link twist

Link	θ_i	d_i	a_i	α_i
1	θ_1	d_1	0	0
2	0	d_2	0	$-\frac{\pi}{2}$
3	0	d_3	0	0

Denavit-Hartenberg Convention



Joints:
Revolute: rotation around z-axis
Prismatic: motion along z-axis

The axis x_i intersects and is perpendicular to axis z_{i-1}

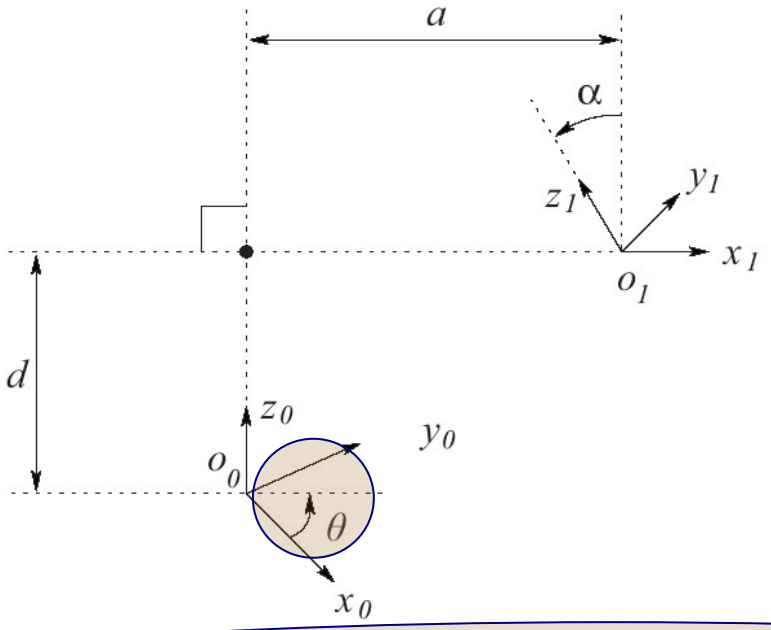
$\boldsymbol{\theta_i}$ = joint angle: the angle between x_{i-1} and x_i measured about z_{i-1}.
θ_i is variable if joint i is revolute

$\boldsymbol{d_i}$ = link offset: distance along z_{i-1} from O_{i-1} to the intersection of the x_i and z_{i-1} axes. d_i is variable if joint i is prismatic

$\boldsymbol{a_i}$ = link length: distance along x_i from O_i to the intersection of the x_i and z_{i-1} axes

$\boldsymbol{\alpha_i}$ = link twist: the angle between z_{i-1} and z_i measured about x_i

Denavit-Hartenberg Convention



$$\begin{pmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathrm{Rot}_{z,\theta_i}$$

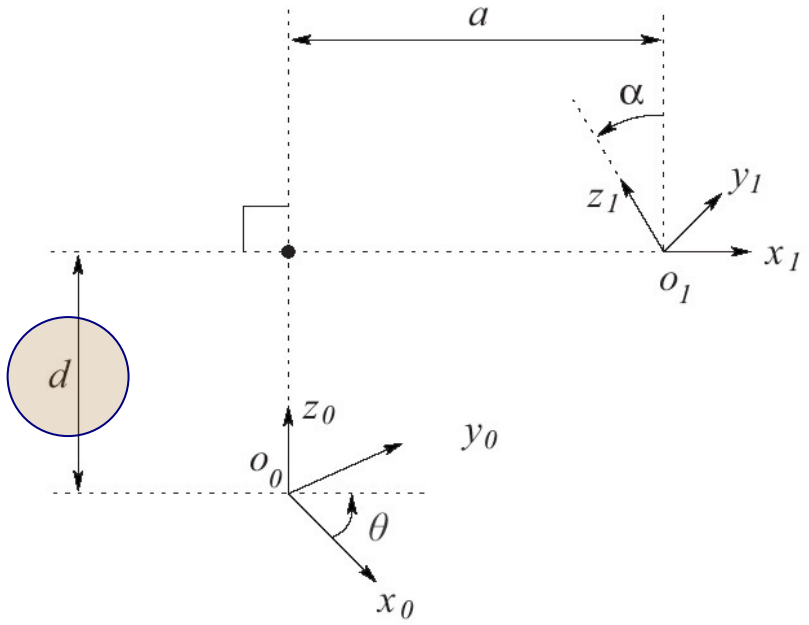
$\boldsymbol{\theta_i}$ = joint angle: the angle between x_{i-1} and x_i measured about z_{i-1}. θ_i is variable if joint i is revolute

$\boldsymbol{d_i}$ = link offset: distance along z_{i-1} from O_{i-1} to the intersection of the x_i and z_{i-1} axes. d_i is variable if joint i is prismatic

$\boldsymbol{a_i}$ = link length: distance along x_i from O_i to the intersection of the x_i and z_{i-1} axes

$\boldsymbol{\alpha_i}$ = link twist: the angle between z_{i-1} and z_i measured about x_i

Denavit-Hartenberg Convention



$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Trans_{z,d_i}

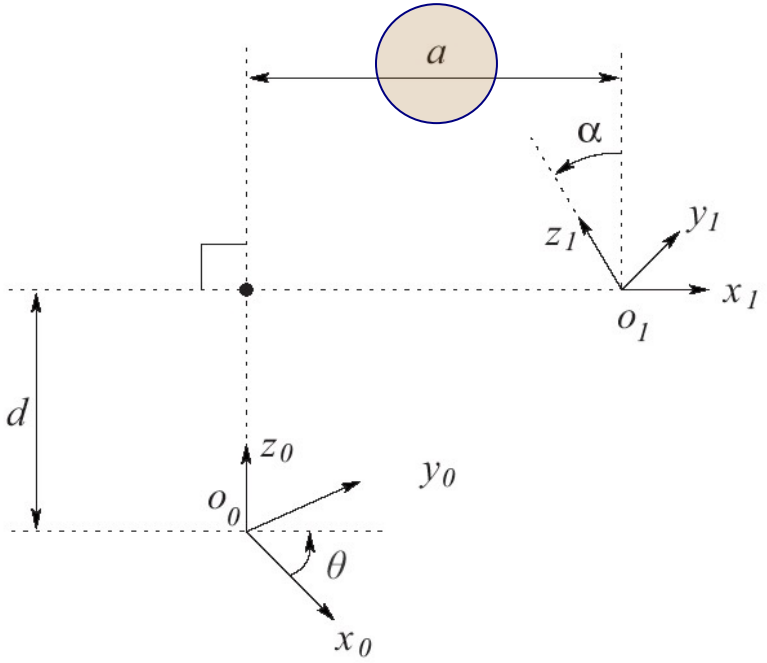
$\boldsymbol{\theta_i}$ = joint angle: the angle between x_{i-1} and x_i measured about z_{i-1}.
θ_i is variable if joint i is revolute

$\boldsymbol{d_i}$ = link offset: distance along z_{i-1} from O_{i-1} to the intersection of the x_i and z_{i-1} axes. d_i is variable if joint i is prismatic

$\boldsymbol{a_i}$ = link length: distance along x_i from O_i to the intersection of the x_i and z_{i-1} axes

$\boldsymbol{\alpha_i}$ = link twist: the angle between z_{i-1} and z_i measured about x_i

Denavit-Hartenberg Convention



$$\begin{pmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Trans_{x,a_i}

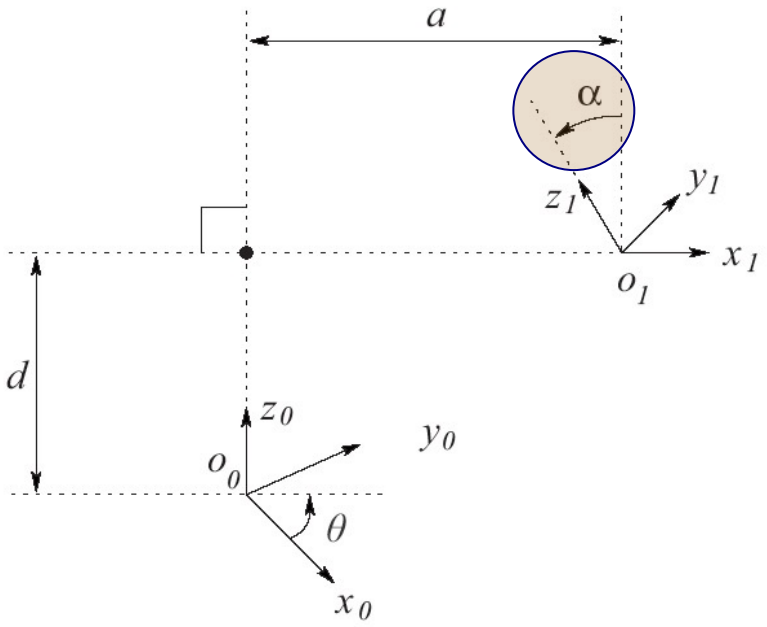
$\boldsymbol{\theta_i}$ = joint angle: the angle between x_{i-1} and x_i measured about z_{i-1}.
θ_i is variable if joint i is revolute

$\boldsymbol{d_i}$ = link offset: distance along z_{i-1} from O_{i-1} to the intersection of the x_i and z_{i-1} axes. d_i is variable if joint i is prismatic

$\boldsymbol{a_i}$ = link length: distance along x_i from O_i to the intersection of the x_i and z_{i-1} axes

$\boldsymbol{\alpha_i}$ = link twist: the angle between z_{i-1} and z_i measured about x_i

Denavit-Hartenberg Convention



$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\alpha_i} & -s_{\alpha_i} & 0 \\ 0 & s_{\alpha_i} & c_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathrm{Rot}_{x,\alpha_i}$$

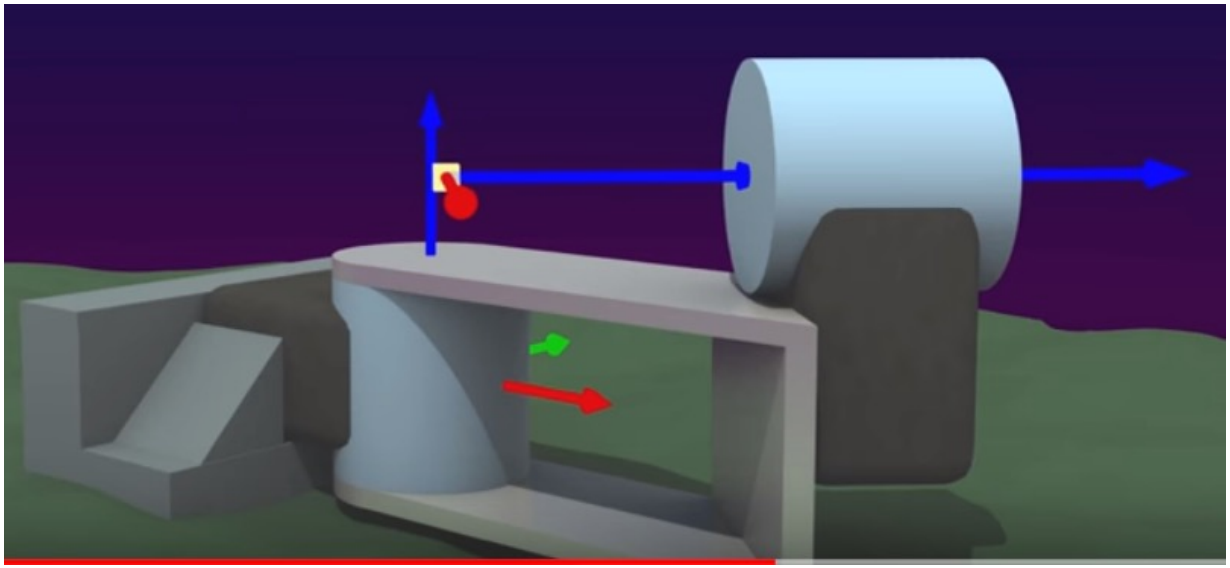
$\boldsymbol{\theta_i}$ = joint angle: the angle between x_{i-1} and x_i measured about z_{i-1}.
θ_i is variable if joint i is revolute

$\boldsymbol{d_i}$ = link offset: distance along z_{i-1} from O_{i-1} to the intersection of the x_i and z_{i-1} axes. d_i is variable if joint i is prismatic

$\boldsymbol{a_i}$ = link length: distance along x_i from O_i to the intersection of the x_i and z_{i-1} axes

$\boldsymbol{\alpha_i}$ = link twist: the angle between z_{i-1} and z_i measured about x_i

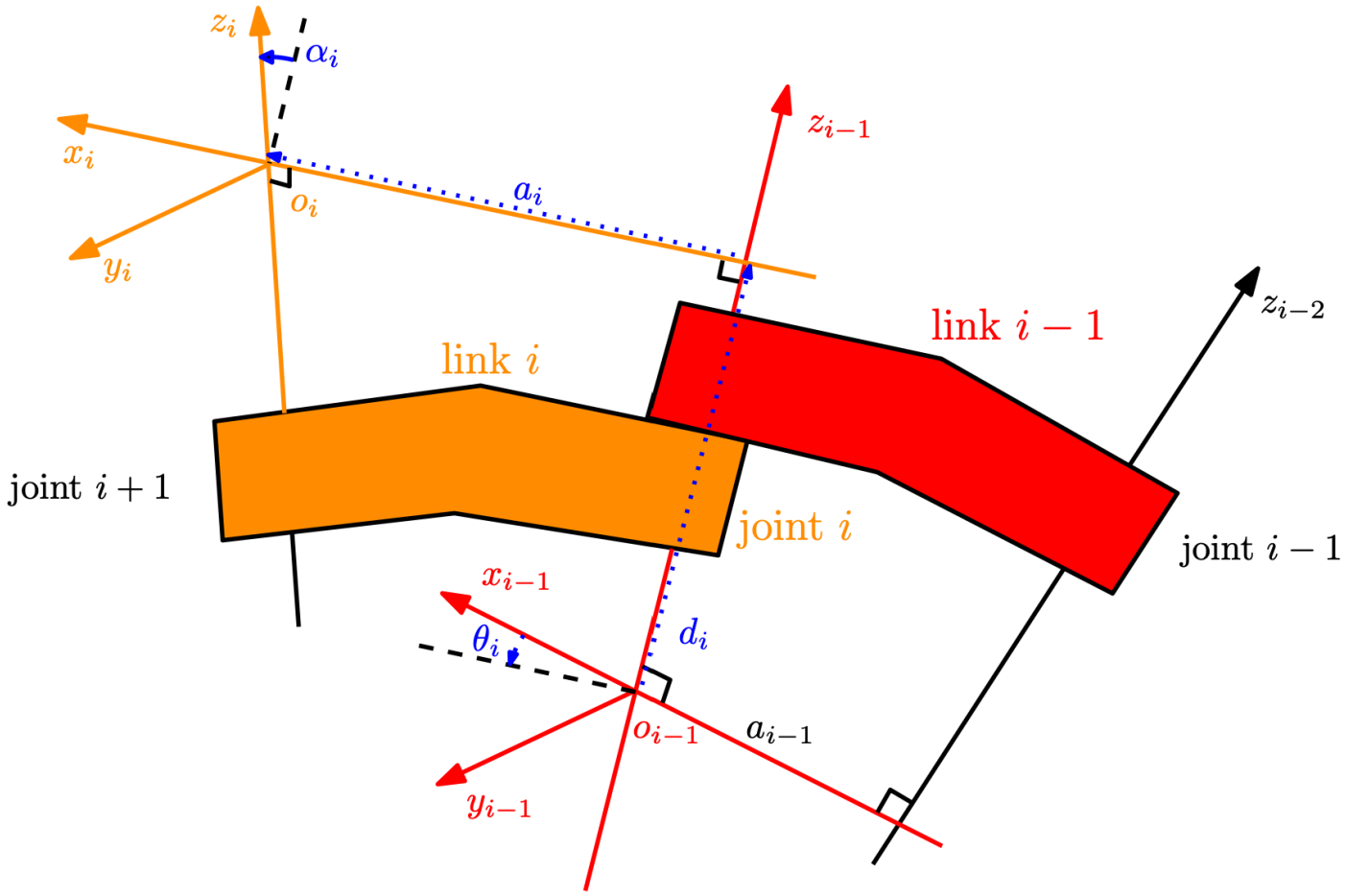
Denavit-Hartenberg – Illustration



https://www.youtube.com/watch?v=rA9tm0gTIn8

(Warning: Good illustration but ”wrong direction/sign” of theta!)

A General Configuration



D-H Representation: A-matrices

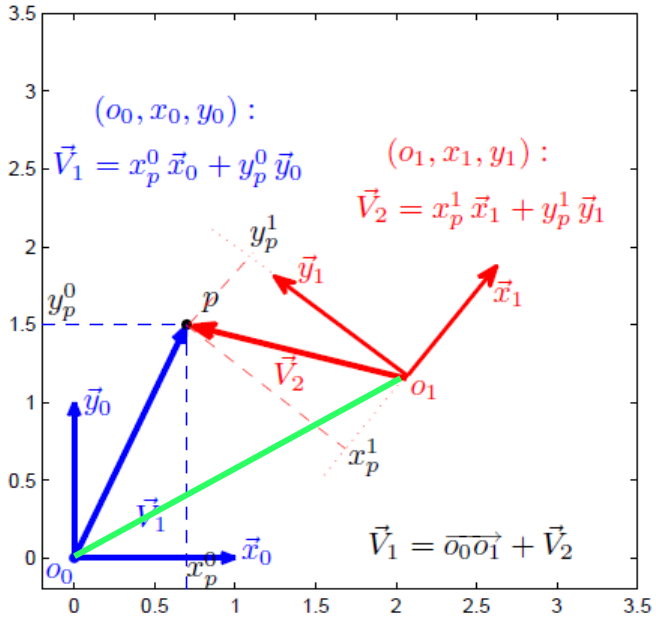
$$A_i^{i-1} = A_i = \mathrm{Rot}_{z,\theta_i}\,\mathrm{Trans}_{z,d_i}\,\mathrm{Trans}_{x,a_i}\,\mathrm{Rot}_{x,\alpha_i}$$

$$\begin{pmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\alpha_i} & -s_{\alpha_i} & 0 \\ 0 & s_{\alpha_i} & c_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Recap: Homogeneous Transformation

Combine both rotation and translation in one 4x4 matrix



$$\underbrace{\begin{bmatrix} p^0 \\ 1 \end{bmatrix}}_{P^0} = \begin{bmatrix} x_p^0 \\ y_p^0 \\ z_p^0 \\ 1 \end{bmatrix} = \begin{bmatrix} (x_1^0)_x & (y_1^0)_x & (z_1^0)_x & (o_1^0)_x \\ (x_1^0)_y & (y_1^0)_y & (z_1^0)_y & (o_1^0)_y \\ (x_1^0)_z & (y_1^0)_z & (z_1^0)_z & (o_1^0)_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_p^1 \\ y_p^1 \\ z_p^1 \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} R_1^0 & o_1^0 \\ 0_{1\times 3} & 1 \end{bmatrix}}_{H_1^0} \underbrace{\begin{bmatrix} p^1 \\ 1 \end{bmatrix}}_{P^1}$$

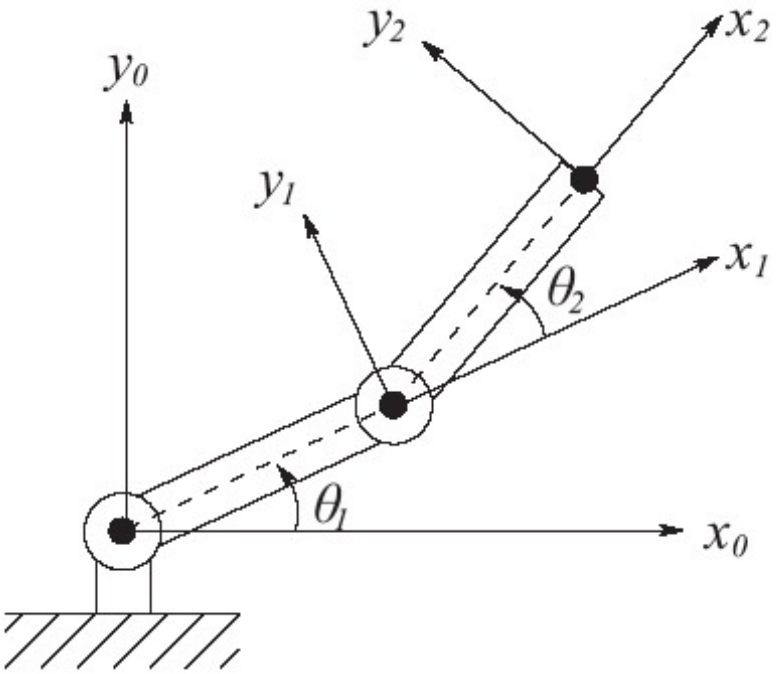
D-H Representation: ORDER MATTERS!

$$A_i^{i-1} = A_i = \mathrm{Rot}_{z,\theta_i}\,\mathrm{Trans}_{z,d_i}\,\mathrm{Trans}_{x,a_i}\,\mathrm{Rot}_{x,\alpha_i}$$

$$\begin{pmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\alpha_i} & -s_{\alpha_i} & 0 \\ 0 & s_{\alpha_i} & c_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} c_{\theta_i} & -s_{\theta_i} c_{\alpha_i} & s_{\theta_i} s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i} c_{\alpha_i} & -c_{\theta_i} s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Two-Link Planar Manipulator Revisited



[Discuss 2 min]

Link	a_i	α_i	d_i	θ_i
1	a_1	0	0	θ_1
2	a_2	0	0	θ_2

$\boldsymbol{\theta_i}$ = joint angle: the angle between x_{i-1} and x_i measured about z_{i-1}.
θ_i is variable if joint i is revolute

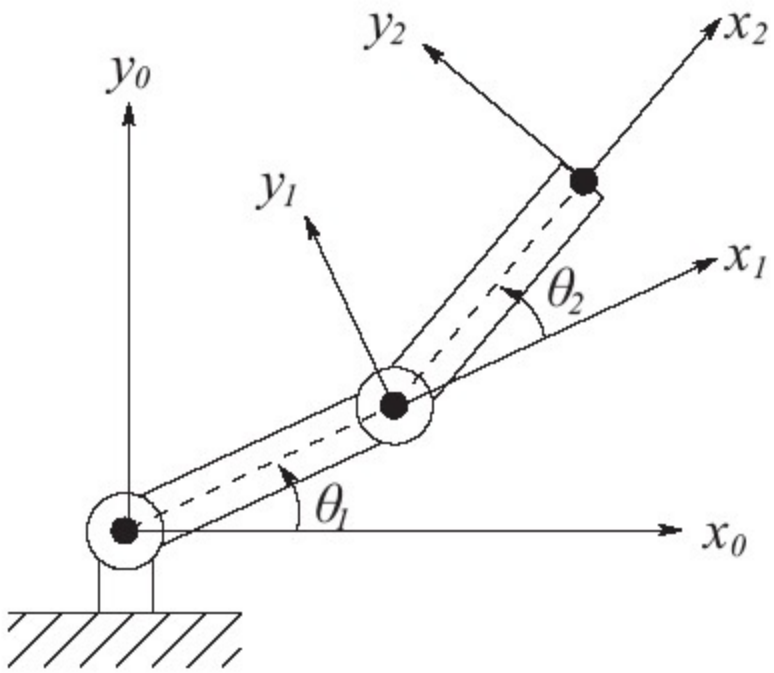
$\boldsymbol{d_i}$ = link offset: distance along z_{i-1} from O_{i-1} to the intersection of the x_i and z_{i-1} axes. d_i is variable if joint i is prismatic

$\boldsymbol{a_i}$ = link length: distance along x_i from O_i to the intersection of the x_i and z_{i-1} axes

$\boldsymbol{\alpha_i}$ = link twist: the angle between z_{i-1} and z_i measured about x_i

LUND UNIVERSITY

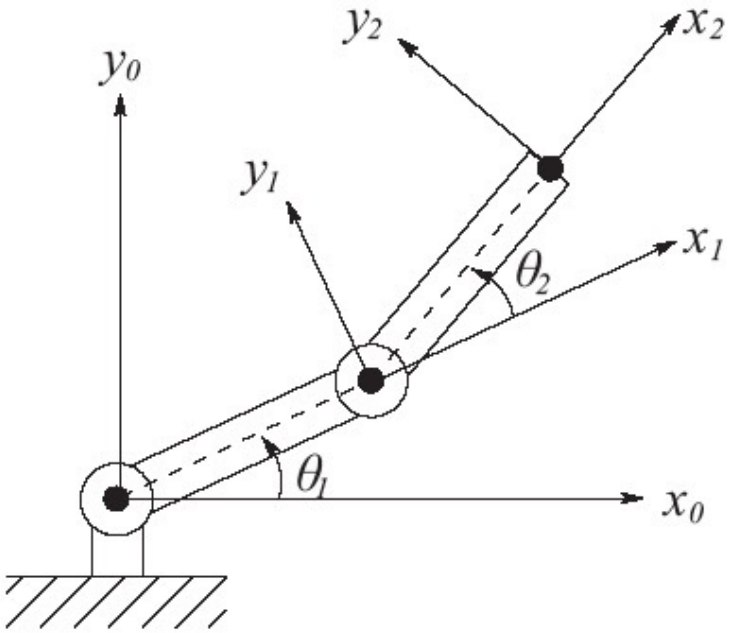
Forward Kinematics



$$A_1 = \begin{pmatrix} c_{\theta_1} & -s_{\theta_1} & 0 & a_1 c_{\theta_1} \\ s_{\theta_1} & c_{\theta_1} & 0 & a_1 s_{\theta_1} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$A_2 = \begin{pmatrix} c_{\theta_2} & -s_{\theta_2} & 0 & a_2 c_{\theta_2} \\ s_{\theta_2} & c_{\theta_2} & 0 & a_2 s_{\theta_2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Forward Kinematics (cont'd)



$$T_2^0 = A_1 A_2 = \begin{pmatrix} c_{\theta_1+\theta_2} & -s_{\theta_1+\theta_2} & 0 & a_1 c_{\theta_1} + a_2 c_{\theta_1+\theta_2} \\ s_{\theta_1+\theta_2} & c_{\theta_1+\theta_2} & 0 & a_1 s_{\theta_1} + a_2 s_{\theta_1+\theta_2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Summary

Summary

- Course administration
- Kinematics of robots [Chapter 2 in Lecture Notes]
  - Homogeneous transformation matrices
  - Denavit-Hartenberg (DH) convention
- Forward kinematics of a serial-kinematic robot manipulator
  - Step-by-step introduction of frames along the robot in a systematic way

LUND
UNIVERSITY